RTL Design Sherpa

RTL AMBA APB4 Module Reference — Rev 0.90

July 2026

Table of Contents

1 APB (Advanced Peripheral Bus) Modules....15
1.1 Overview....15
1.1.1 Protocol Scope: APB4, not APB5....16
1.2 Module Categories....16
1.2.1 Core Protocol Components....16
1.2.2 Testbench Utilities....17
1.2.3 Interconnect Components....17
1.2.4 Clock-Gated Variants....17
1.3 Key Features....18
1.3.1 APB Protocol Support....18
1.3.2 Clock Domain Crossing....18
1.3.3 Monitoring and Debug....18
1.3.4 Testbench Integration....18
1.4 Quick Start....18
1.4.1 Using APB Master....18
1.4.2 Using APB Slave....19
1.5 Testing....20
1.5.1 WaveDrom Tests....20
1.5.2 Test Coverage Gaps....21
1.6 Protocol Details....21
1.6.1 APB Transfer Phases....21
1.6.2 Signal Descriptions....21

1.7 Design Notes..... 22
1.7.1 Command/Response Architecture..... 22
1.7.2 Monitor Bus Protocol..... 22
1.8 Related Documentation..... 23
1.9 References..... 23
1.9.1 Specifications..... 23
1.9.2 Source Code..... 23
1.9.3 Test Documentation..... 23
1.10 Navigation..... 24
2 APB Crossbar Specification..... 24
2.1 Table of Contents..... 24
2.2 Overview..... 24
2.2.1 Key Features..... 25
2.2.2 Design Philosophy..... 25
2.3 Architecture..... 26
2.3.1 Component Hierarchy..... 26
2.3.2 Data Flow..... 26
2.4 Available Modules..... 27
2.4.1 Feature Comparison..... 28
2.4.2 Latency..... 28
2.4.3 Selecting a Crossbar..... 29
2.5 Interface Specification..... 29
2.5.1 Parameters..... 29
2.5.2 Port List..... 30

2.5.3 Signal Descriptions....31
2.6 Address Mapping....31
2.6.1 Default Address Map....31
2.6.2 Example: Default Configuration....32
2.6.3 Custom Address Map....32
2.6.4 Alignment Requirement....32
2.6.5 Decode Errors: Unmapped Addresses Stall....33
2.7 Arbitration....33
2.7.1 Round-Robin Arbitration....33
2.7.2 Grant Persistence....34
2.7.3 Arbitration Example....34
2.7.4 Arbitration Latency....34
2.8 Timing Diagrams....35
2.8.1 Basic Write Transaction (No Wait States)....35
2.8.2 Basic Read Transaction (No Wait States)....35
2.8.3 Write with Wait States....35
2.8.4 Arbitration with Contention....36
2.9 Integration Examples....36
2.9.1 Example 1: Simple 1-to-4 Crossbar....36
2.9.2 Example 2: Multi-Master System (2-to-4)....38
2.9.3 Example 3: Custom Configuration (3x8 Crossbar)....39
2.10 Test Results....39
2.10.1 Test Coverage....39
2.10.2 Formal Verification....39

2.10.3 Test Scenarios..... 40
2.10.4 Running Tests..... 40
2.10.5 Test Results Summary..... 40
2.11 Known Limitations..... 40
2.11.1 Current Limitations..... 40
2.11.2 Future Enhancements..... 41
2.11.3 Known Issues..... 41
2.12 Related Documentation..... 42
2.13 Revision History..... 42
2.14 Navigation..... 43
3 apb_master..... 43
3.1 Overview..... 43
3.2 Module Declaration..... 43
3.3 Parameters..... 44
3.4 Ports..... 45
3.4.1 Clock and Reset..... 45
3.4.2 APB Master Interface..... 45
3.4.3 Command Interface..... 46
3.4.4 Response Interface..... 46
3.5 Functionality..... 47
3.5.1 APB Protocol Implementation..... 47
3.5.2 Command Processing Flow..... 47
3.5.3 Key Features..... 47
3.6 Timing Characteristics..... 47

3.6.1 APB Transaction Timing....47
3.6.2 Performance Metrics....48
3.7 Waveforms....48
3.7.1 Scenario 1: Basic Write Transaction....48
3.7.2 Scenario 2: Basic Read Transaction....49
3.7.3 Scenario 3: Back-to-Back Writes....49
3.7.4 Scenario 4: Back-to-Back Reads....50
3.7.5 Scenario 5: Write-to-Read Transition....50
3.7.6 Scenario 6: Read-to-Write Transition....50
3.8 Usage Examples....51
3.8.1 Basic APB Master Configuration....51
3.8.2 Register Write Sequence....51
3.8.3 Register Read Sequence....52
3.8.4 Burst Operation Example....52
3.9 Integration with APB Interconnect....53
3.9.1 APB Crossbar Integration....53
3.10 Advanced Features....54
3.10.1 Protection Attributes (PPROT)....54
3.10.2 Write Strobe Support....55
3.10.3 Error Handling....55
3.11 Performance Optimization....55
3.11.1 FIFO Depth Selection....55
3.11.2 Clock Gating....56
3.12 Synthesis Considerations....56

3.12.1 Area Optimization....56
3.12.2 Timing Optimization....57
3.12.3 Power Optimization....57
3.13 Verification Notes....57
3.13.1 Protocol Compliance....57
3.13.2 Interface Verification....57
3.13.3 Performance Verification....57
3.14 Related Modules....57
3.15 Navigation....58
4 APB Master Interface (Clock-Gated)....58
4.1 Quick Reference....58
4.2 Summary....58
4.3 Additional Parameters....59
4.4 Additional Ports....59
4.4.1 Wake-Up Condition....59
4.4.2 Wake-Up Latency....60
4.5 Quick Usage....60
4.6 Documentation....60
4.7 Navigation....61
5 apb_master_stub....61
5.1 Overview....61
5.2 Module Declaration....61
5.3 Parameters....62
5.4 Ports....63

5.4.1 Clock and Reset 63
5.4.2 APB Master Interface 63
5.4.3 Packed Command Interface 64
5.4.4 Packed Response Interface 64
5.5 Functional Description 64
5.5.1 Packet Interface 64
5.5.2 First/Last Framing 64
5.5.3 Packet Format Is Not Symmetric With apb_slave_stub 65
5.5.4 Operation 65
5.6 Usage Example 65
5.7 Design Notes 66
5.7.1 Testbench Usage 66
5.7.2 Packed Interface Benefits 66
5.8 Related Modules 66
5.9 References 66
5.10 Navigation 66
6 apb_slave 67
6.1 Overview 67
6.2 Module Declaration 67
6.3 Parameters 68
6.4 Ports 69
6.4.1 Clock and Reset 69
6.4.2 APB Slave Interface 69
6.4.3 Command Interface 69

6.4.4 Response Interface....70
6.5 Functionality....70
6.5.1 APB Protocol Implementation....70
6.5.2 Command/Response Flow....71
6.5.3 Buffering Architecture....71
6.5.4 Key Features....71
6.6 State Machine Operation....72
6.6.1 State Transitions....72
6.6.2 Orphan-Response Guard....72
6.6.3 APB Transaction Timing....72
6.7 Timing Characteristics....72
6.7.1 Transaction Latency....72
6.7.2 Performance Metrics....73
6.8 Waveforms....73
6.8.1 Scenario 1: Basic Write Transaction....74
6.8.2 Scenario 2: Basic Read Transaction....74
6.8.3 Scenario 3: Back-to-Back Writes....74
6.8.4 Scenario 4: Back-to-Back Reads....74
6.8.5 Scenario 5: Write-to-Read Transition....75
6.8.6 Scenario 6: Read-to-Write Transition....75
6.8.7 Scenario 7: Error Response....75
6.9 Usage Examples....76
6.9.1 Basic Register Block Interface....76
6.9.2 Register Block Backend Implementation....76

6.9.3 Memory Interface Example....78
6.10 Advanced Integration Patterns....79
6.10.1 Multi-Register Bank System....79
6.10.2 Error Handling and Status Reporting....80
6.11 Performance Optimization....82
6.11.1 Buffer Depth Selection....82
6.11.2 Clock Domain Optimization....82
6.12 Synthesis Considerations....83
6.12.1 Area Optimization....83
6.12.2 Timing Optimization....83
6.12.3 Power Optimization....83
6.13 Verification Notes....83
6.13.1 Protocol Compliance....83
6.13.2 Buffer Verification....83
6.13.3 Backend Integration....83
6.14 Related Modules....84
6.15 Navigation....84
7 APB Slave CDC....84
7.1 Overview....84
7.1.1 Key Features....84
7.2 Module Interface....85
7.3 Parameters....86
7.4 Clock Domains....86
7.4.1 APB Domain (pclk)....86

7.4.2 AXI Domain (aclk)....86
7.5 CDC Implementation....87
7.5.1 Structure....87
7.5.2 Maximum Clock Ratio....87
7.5.3 Reset Behavior....87
7.5.4 Why Not the Previous 2-Phase Handshake....88
7.5.5 Timing Constraints....88
7.6 Waveforms....88
7.6.1 Scenario 1: Write Transaction with CDC....88
7.6.2 Scenario 2: Read Transaction with CDC....89
7.6.3 Scenario 3: Back-to-Back Writes with CDC....89
7.6.4 Scenario 4: Back-to-Back Reads with CDC....90
7.6.5 Scenario 5: Write-to-Read Transition with CDC....90
7.6.6 Scenario 6: Read-to-Write Transition with CDC....90
7.6.7 Scenario 7: Error Response with CDC....91
7.7 Usage Example....91
7.8 References....92
7.9 Navigation....93
8 APB Slave with CDC (Clock-Gated)....93
8.1 Quick Reference....93
8.2 Summary....93
8.3 Additional Parameters....94
8.4 Additional Ports....94
8.5 Quick Usage....95

8.6 Documentation....95
8.7 Navigation....96
9 APB Slave Interface (Clock-Gated)....96
9.1 Overview....96
9.1.1 Key Differences from Base Module....96
9.2 When to Use Clock-Gated Variant....96
9.3 Clock Gating Parameters....97
9.4 Clock Gating Ports....97
9.5 Usage Examples....97
9.5.1 Example 1: Aggressive Gating (Bursty Traffic)....97
9.5.2 Example 2: Conservative Gating....98
9.5.3 Example 3: Clock Gating Disabled (Functional Verification)....98
9.6 Clock Gating Architecture....99
9.6.1 Single Gating Domain....99
9.6.2 Wake-Up Condition....99
9.6.3 Gating State Machine....99
9.6.4 Wake-Up Latency....99
9.7 Power Savings....100
9.7.1 Gating Status Signal....100
9.8 Verification Considerations....100
9.8.1 Clock Gating in Simulation....100
9.8.2 Power Analysis Verification....100
9.9 Synthesis Considerations....101
9.9.1 FPGA Implementations....101

9.9.2 ASIC Implementations..... 101
9.10 Related Modules..... 101
9.11 See Also..... 101
9.12 Navigation..... 102
10 apb_slave_stub..... 102
10.1 Overview..... 102
10.2 Module Declaration..... 102
10.3 Parameters..... 103
10.4 Ports..... 103
10.4.1 Clock and Reset..... 103
10.4.2 APB Slave Interface..... 104
10.4.3 Packed Command Interface..... 104
10.4.4 Packed Response Interface..... 104
10.5 Functional Description..... 105
10.5.1 Packet Interface..... 105
10.5.2 Packet Format Is Not Symmetric With apb_master_stub..... 105
10.5.3 Difference From apb_slave..... 106
10.5.4 Operation..... 106
10.6 Usage Example..... 106
10.7 Design Notes..... 107
10.7.1 Testbench Usage..... 107
10.7.2 Response Generation..... 107
10.7.3 Packed Interface Benefits..... 107
10.8 Related Modules..... 107

10.9 References....108
10.10 Navigation....108
11 apb_xbar_thin....108
11.1 Naming Note....108
11.1.1 Choosing Between the Two Families....109
11.2 Module Declaration....109
11.3 Parameters....111
11.4 Ports....111
11.4.1 Clock and Reset....111
11.4.2 Configuration Interface....111
11.4.3 Master Interfaces (Input Side)....112
11.4.4 Slave Interfaces (Output Side)....113
11.4.5 A Note on the m_/s_ Prefixes....114
11.5 Architecture....114
11.5.1 Crossbar Topology....114
11.5.2 Key Components....114
11.5.3 Data Flow....114
11.6 Functionality....115
11.6.1 Address Decoding....115
11.6.2 Weighted Round-Robin Arbitration with ACK Mode....115
11.7 Timing Characteristics....116
11.7.1 Latency....116
11.7.2 Arbitration Turnaround....116
11.7.3 Throughput....116

11.8 Usage Example....117
11.8.1 3 Masters, 4 Slaves with a Sparse Address Map....117
11.8.2 Runtime Address Map Reprogramming....118
11.9 Known Limitations....119
11.9.1 Unmapped Addresses Stall the Bus....119
11.9.2 Other Limitations....119
11.10 Synthesis Considerations....120
11.10.1 Area....120
11.10.2 Timing....120
11.11 Verification....121
11.11.1 Formal....121
11.11.2 Simulation....121
11.11.3 Suggested Coverage....121
11.12 Related Modules....121
11.13 Navigation....122

1 APB (Advanced Peripheral Bus) Modules

Location: `rtl/amba/apb/` **Test Location:** `val/amba/` **Status:** ✓ Production Ready

---

1.1 Overview

The APB subsystem provides a complete implementation of the ARM AMBA 4 APB (Advanced Peripheral Bus) protocol, including masters, slaves, monitors, interconnect components, and testbench utilities.

APB is a simple, low-power peripheral bus designed for connecting low-bandwidth peripherals to a system bus. It uses a simple two-cycle handshake protocol with minimal control signals.

1.1.1 Protocol Scope: APB4, not APB5

Every module in `rtl/amba/apb/` implements **AMBA 4 APB (APB4)**: PSEL, PENABLE, PREADY, PADDR, PWRITE, PWDATA, PSTRB, PPROT, PRDATA, PSLVERR.

The APB5 additions – PWAKEUP, PAUSER/PWUSER/PRUSER/PBUSER, and the optional parity signals – are **not** present on these modules. They live in a separate module family:

Family	RTL	Documentation
APB4 (this book)	`rtl/amba/apb/`	`docs/markdown/RTLAmba/apb/`
APB5	`rtl/amba/apb5/`	APB5 Modules

Use `apb5_slave` / `apb5_master` (and their `_cg` / `_cdc` variants) when APB5 signalling is required. The two families are otherwise architecturally identical.

1.2 Module Categories

1.2.1 Core Protocol Components

Module	Description	Documentation	Status
apb_master	Full-featured APB master with command/response interface	apb_master.md	✓ Documented
apb_slave	Complete APB slave with buffered cmd/rsp interface	apb_slave.md	✓ Documented
apb_slave_cdc	APB slave with clock domain crossing support	apb_slave_cdc.md	✓ Documented
apb_monitor	Transaction monitoring with 128-bit monitor bus + 64-bit timestamp	apb_monitor.md	✓ Documented

Note: `apb_monitor.sv` lives in `rtl/amba/monitor/` (with the rest of the monitor family), not in `rtl/amba/apb/`. Its specification is part of the Monitor book.

1.2.2 Testbench Utilities

Module	Description	Documentation	Status
apb_master_stub	Lightweight APB master for testbench integration	apb_master_stub.md	✓ Documented
apb_slave_stub	Lightweight APB slave for testbench integration	apb_slave_stub.md	✓ Documented

1.2.3 Interconnect Components

Module	Description	Documentation	Status
apb_xbar_1to1 / **2to1** / **1to4** / **2to4**	Generated fixed-configuration crossbars built from `apb_slave` + `apb_master`	apb_crossbar.md	✓ Documented
apb_xbar_thin	Fully parameterized M×S combinational crossbar with weighted round-robin	apb_xbar.md	✓ Documented

Note: The crossbar RTL, generator, and testbenches live in the `apb_xbar` component area (`projects/components/apb_xbar/`), not under `rtl/amba/apb/`. They are documented here because they are built entirely from the APB4 primitives in this directory.

1.2.4 Clock-Gated Variants

Each of these wraps its base module in `amba_clock_gate_ctrl` and is functionally identical to it. They add `cfg_cg_enable` / `cfg_cg_idle_count` inputs and an `apb_clock_gating` status output.

Module	Base Module	Documentation	Status
apb_master_cg	`apb_master`	apb_master_cg.md	✓ Documented
apb_slave_cg	`apb_slave`	apb_slave_cg.md	✓ Documented
apb_slave_cdc_cg	`apb_slave_cdc`	apb_slave_cdc_cg.md	✓ Documented

1.3 Key Features

1.3.1 APB Protocol Support

- ✓ **Full APB4 Compliance:** Complete AMBA 4 APB protocol implementation
- ✓ **PSTRB Support:** Byte-lane strobes for partial writes
- ✓ **PPROT Support:** Protection attributes for security-aware systems
- ✓ **Error Handling:** PSLVERR support for error responses
- ✓ **APB5 Available Separately:** See `rtl/amba/apb5/` for `PWAKEUP`, the `P*USER` sidebands, and optional parity

1.3.2 Clock Domain Crossing

- ✓ **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains
- ✓ **Safe CDC:** Gray-pointer asynchronous FIFOs (`gaxi_fifo_async`) in both directions
- ✓ **Independent Frequencies:** Backend can run faster or slower than APB
- ✓ **Independent Resets:** Each domain may be reset alone without desynchronizing the link

1.3.3 Monitoring and Debug

- ✓ **Transaction Monitoring:** Real-time protocol monitoring
- ✓ **128-bit Monitor Bus:** Standardized packet format plus a 64-bit side-band timestamp
- ✓ **Error Detection:** Protocol violations, timeout detection
- ✓ **Performance Tracking:** Transaction counting, latency measurement

1.3.4 Testbench Integration

- ✓ **Packed Interfaces:** Simplified testbench connectivity
- ✓ **Stub Modules:** Lightweight wrappers for CocoTB integration
- ✓ **WaveDrom Support:** Automated waveform generation

1.4 Quick Start

1.4.1 Using APB Master

```
apb_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
```

```
) u_apb_master (
    .aclk           (clk),
    .aresetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),

    // APB master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr)
);
```

1.4.2 Using APB Slave

```
apb_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_slave (
    .aclk           (clk),
    .aresetn        (resetn),

    // APB slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
```

```
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

1.5 Testing

All APB modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB tests
pytest val/amba/test_apb*.py -v

# Run specific module tests
pytest val/amba/test_apb_master.py -v
pytest val/amba/test_apb_slave.py -v
pytest val/amba/test_apb_slave_cdc.py -v
pytest val/amba/test_apb_monitor.py -v

# Clock-gated CDC variant
pytest val/amba/test_apb_slave_cdc_cg.py -v

# Generated crossbars (component area)
pytest projects/components/apb_xbar/dv/tests/ -v

# Run with waveform generation
env ENABLE_WAVEDROM=1 pytest val/amba/test_apb_slave_wavedrom.py -v
```

1.5.1 WaveDrom Tests

Several modules have dedicated WaveDrom tests that generate detailed timing diagrams:

- `test_apb_slave_wavedrom.py::test_comprehensive_apb_slave` - APB slave protocol waveforms
- `test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom` - CDC timing diagrams

1.5.2 Test Coverage Gaps

The following APB4 modules have no dedicated test at `val/amba/`. They are exercised indirectly (stubs through the generated crossbars and the AXI4-to-APB bridge; the clock-gated wrappers through their base modules), and direct tests exist for the APB5 equivalents:

Module	Direct APB4 test	APB5 equivalent
`apb_master_stub`	None	`test_apb5_master_stub.py`
`apb_slave_stub`	None	`test_apb5_slave_stub.py`
`apb_master_cg`	None	`test_apb5_master_cg.py`
`apb_slave_cg`	None	`test_apb5_slave_cg.py`

Generated waveforms are stored in: - JSON format: `_wavedrom/` - SVG images: `_wavedrom_svg/`

1.6 Protocol Details

1.6.1 APB Transfer Phases

APB uses a simple two-phase protocol:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer
2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

1.6.2 Signal Descriptions

Signal	Direction	Description
PCLK	Input	APB clock

Signal	Direction	Description
PRESETn	Input	Active-low reset
PSEL	Master→Slave	Slave select
PENABLE	Master→Slave	Enable (ACCESS phase indicator)
PREADY	Slave→Master	Transfer complete
PADDR	Master→Slave	Address bus
PWRITE	Master→Slave	Write enable (1=write, 0=read)
PWDATA	Master→Slave	Write data
PSTRB	Master→Slave	Byte lane strobes
PPROT	Master→Slave	Protection attributes
PRDATA	Slave→Master	Read data
PSLVERR	Slave→Master	Error response

1.7 Design Notes

1.7.1 Command/Response Architecture

All APB modules use a command/response interface pattern for backend integration:

Command Interface (Master → Backend or APB → Backend): - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes

Response Interface (Backend → Master or Backend → APB): - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag

This separation enables: - Clean clock domain crossing - Buffering and pipelining - Easy testbench integration - Backend processing flexibility

1.7.2 Monitor Bus Protocol

The APB monitor outputs standardized 128-bit `monitor_packet_t` records, paired with a 64-bit side-band timestamp:

```
[127:124] - Packet Type    (4 bits)
[123:109] - Reserved       (15 bits, forward-compat slack)
[108:105] - Protocol       (4 bits)
[104:97]  - Event Code     (8 bits)
```

```
[96:88]   - Channel ID     (9 bits)
[87:72]   - Agent ID       (16 bits)
[71:64]   - Unit ID        (8 bits)
[63:0]    - Event Data     (64 bits)
```

See apb_monitor.md for detailed packet format.

1.8 Related Documentation

- **APB5 Modules** - APB5 family (PWAKEUP, P*USER, parity)
- **AXI4 Modules** - Full AXI4 protocol components
- **AXIL4 Modules** - AXI4-Lite components
- **AXIS4 Modules** - AXI4-Stream components
- **GAXI Modules** - Generic AXI utilities

1.9 References

1.9.1 Specifications

- **ARM IHI 0024C – AMBA APB Protocol Specification, Version 2.0 (defines APB4: adds PSTRB and PPROT). This is the specification these modules implement.**
- **ARM IHI 0024E – AMBA APB Protocol Specification, Issue E (APB5: PWAKEUP, P*USER sidebands, parity). Applies to rtl/amba/apb5/, not to this family.**

Naming caution: ARM's *document* version 2.0 (IHI 0024C) is the APB4 specification. The older

APB3 protocol is IHI 0024B. Citing “APB Protocol Specification v2.0” without the IHI number is ambiguous.

1.9.2 Source Code

- RTL: `rtl/amba/apb/`
- Tests: `val/amba/test_apb*.py`
- Framework: `bin/TBClasses/components/apb/`

1.9.3 Test Documentation

- APB Master Tests: `val/amba/test_apb_master.py`
- APB Slave Tests: `val/amba/test_apb_slave.py`
- APB CDC Tests: `val/amba/test_apb_slave_cdc.py`
- APB Monitor Tests: `val/amba/test_apb_monitor.py`
- APB Crossbar Tests: `projects/components/apb_xbar/dv/tests/`

Last Updated: 2026-07-19

1.10 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

2 APB Crossbar Specification

Version: 1.1 **Last Updated:** 2026-07-19 **Status:** Production Ready **Location:** `projects/components/apb_xbar/`

2.1 Table of Contents

1. Overview
2. Architecture
3. Available Modules

4. Interface Specification
5. Address Mapping
6. Arbitration
7. Timing Diagrams
8. Integration Examples
9. Test Results
10. Known Limitations

2.2 Overview

The APB crossbar family provides scalable interconnect solutions for connecting multiple APB masters to multiple APB slaves. All crossbar variants are generated using the `apb_xbar_generator.py` tool and share a common architecture based on `apb_slave` and `apb_master` building blocks.

Where things live. The crossbar RTL, generator, and testbenches are in the `apb_xbar` component area, not under `rtl/amba/apb/`:

Artifact	Path
Generated RTL	`projects/components/apb_xbar/rtl/`
Generator	`projects/components/apb_xbar/bin/apb_xbar_generator.py`
Convenience script	`projects/components/apb_xbar/bin/generate_xbars.py`
Testbenches	`projects/components/apb_xbar/dv/tests/`
Formal	`formal/apb_xbar/`

The APB4 primitives they are built from (`apb_slave`, `apb_master`) do live in `rtl/amba/apb/`, which is why this specification is published in the APB4 book.

2.2.1 Key Features

- **Scalable**: Generator takes arbitrary `--masters` / `--slaves`; four pre-generated variants are checked in and tested
- **Standards Compliant**: AMBA 4 APB (APB4). These crossbars carry `PSTRB` and `PPROT`; they do **not** carry APB5 signalling
- **Efficient**: cmd/rsp architecture decouples the two APB sides

- **Fair**: Round-robin arbitration per slave, with grant held until the transaction completes
- **Parameterizable**: Configurable address width, data width, and base address
- **Tested**: CocoTB verification suite plus SymbiYosys formal proofs

2.2.2 Design Philosophy

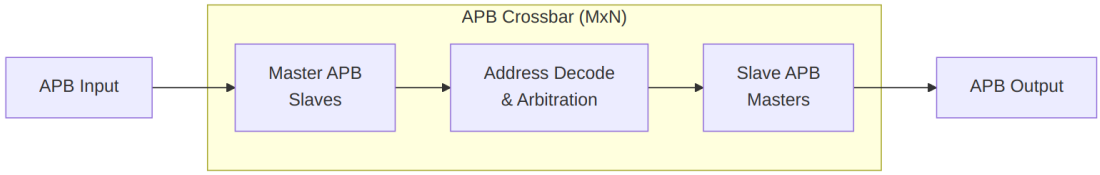
The APB crossbar uses a **protocol conversion** approach:

1. **Master Side**: `apb_slave` modules convert incoming APB transactions to cmd/rsp interface
2. **Crossbar Core**: Address decoding and arbitration logic routes commands to slaves
3. **Slave Side**: `apb_master` modules convert cmd/rsp back to APB transactions

This architecture provides: - Clean separation of concerns - Reusable building blocks - Easy timing closure - Straightforward verification

---

2.3 Architecture



Block Diagram

2.3.1 Component Hierarchy

```
apb_xbar_MtoN
├── apb_slave (M instances)
│   ├── APB protocol → cmd/rsp conversion
│   ├── Command FIFO buffering
│   └── Response routing
├── Address Decoder
│   ├── Slave selection logic (PADDR bit slice)
│   └── Range checking (no decode-error slave -- see Known
Limitations)
├── Arbitration Logic (N instances)
│   ├── Round-robin priority per slave
│   ├── Grant persistence during transaction
│   └── Fairness enforcement
```

```
└── apb_master (N instances)
    ├── cmd/rsp → APB protocol conversion
    ├── APB state machine (IDLE→SETUP→ACCESS)
    └── Response capture
```

2.3.2 Data Flow

Write Transaction:

```
1. Master APB PWRITE=1 → apb_slave
2. apb_slave generates cmd_valid with write data
3. Address decoder selects target slave
4. Arbiter grants access (if available)
5. apb_master converts to APB write
6. Slave APB completes transaction
7. apb_master generates rsp_valid
8. apb_slave routes response back to master
9. Master APB sees PREADY=1
```

Read Transaction:

```
1. Master APB PWRITE=0 → apb_slave
2. apb_slave generates cmd_valid with address
3. Address decoder selects target slave
4. Arbiter grants access (if available)
5. apb_master converts to APB read
6. Slave APB returns data
7. apb_master generates rsp_valid with read data
8. apb_slave routes response back to master
9. Master APB sees PREADY=1 with PRDATA
```

2.4 Available Modules

Module	Masters	Slaves	Primary Use Case	File
apb_xbar_1to1	1	1	Basic passthrough, protocol conversion	projects/components/apb_xbar/rtl/apb_xbar_1to1.sv
apb_xbar_2to1	2	1	Simple arbitration testing	projects/compone

Module	Masters	Slaves	Primary Use Case	File
				nts/apb_xbar/rtl/apb_xbar_2to1.sv
apb_xbar_1to4	1	4	Address decode testing, simple bus	projects/components/apb_xbar/rtl/apb_xbar_1to4.sv
apb_xbar_2to4	2	4	Full crossbar with arbitration + decode	projects/components/apb_xbar/rtl/apb_xbar_2to4.sv
apb_xbar_thin	M	S	Fully parameterized combinational crossbar (different architecture)	projects/components/apb_xbar/rtl/apb_xbar_thin.sv

apb_xbar_thin is **not** a generated variant and does not share this architecture – it is a combinational passthrough with weighted round-robin and no cmd/rsp conversion. See apb_xbar.md.

2.4.1 Feature Comparison

Feature	1to1	2to1	1to4	2to4
Address Decode	✗	✗	√	√
Arbitration	✗	√	✗	√

Feature	1to1	2to1	1to4	2to4
Relative Resources	Minimal	Low	Medium	Medium-High
Relative Max Frequency	Highest	High	High	Medium-High

Resource and frequency columns are relative rankings only. No synthesis has been run for these modules in this repository, so no LUT counts or MHz figures are published. Treat them as an ordering, not as data.

2.4.2 Latency

Every variant, including 1to1, converts APB to cmd/rsp and back: `apb_slave` (master side) → decode/arbitration → `apb_master` (slave side). None of them is a wire.

The dominant cost is that each APB transfer through the crossbar becomes **two** APB transfers – one on the master side and one on the slave side – and the `apb_slave` FSM alone inserts at least two wait states in its ACCESS phase (see apb_slave.md). The structural contributors are:

Stage	Cost
`apb_slave` capture to `cmd_valid`	≥1 cycle (registered FSM)
Address decode	0 cycles (combinational)
Arbitration	1 cycle when contended (`WAIT_GNT_ACK` grant handshake)
`apb_master` SETUP + ACCESS on the downstream slave	≥2 cycles, plus that slave's wait states
Response return through `apb_slave`	≥1 cycle

A single-figure "cycles of latency" number is therefore misleading and is not quoted here. Measure it for your slave's wait-state behaviour if it matters.

2.4.3 Selecting a Crossbar

Use `apb_xbar_1to1` when: - Single master and single slave - Need protocol conversion (APB → cmd/rsp → APB) - Testing or simple verification environments

Use `apb_xbar_2to1` when: - Multiple masters sharing one slave - Need arbitration testing - Simple peripheral with multiple requestors

Use `apb_xbar_1to4` when: - Single master accessing multiple slaves - Address decode testing - Microcontroller with multiple peripherals

Use `apb_xbar_2to4` when: - Multiple masters and multiple slaves - Full system interconnect - Maximum flexibility needed

Need different configuration?

```
# Generate custom crossbar
cd projects/components/apb_xbar/bin
python generate_xbars.py --masters 3 --slaves 8
```

2.5 Interface Specification

2.5.1 Parameters

All crossbar modules support these parameters:

Parameter	Type	Default	Range	Description
`ADDR_WIDTH`	int	32	16-64	APB address bus width
`DATA_WIDTH`	int	32	8, 16, 32, 64	APB data bus width
`BASE_ADDR`	logic[ADDR_WIDTH-1:0]	0x10000000	Any	Base address for slave 0

Derived Parameters: - `STRB_WIDTH = DATA_WIDTH / 8` (automatically calculated) - Slave address regions: 64KB per slave (BASE_ADDR + N*0x10000)

2.5.2 Port List

Clock and Reset:

```
input  logic pclk          // APB clock
input  logic presetn       // APB active-low reset
```

Master Interfaces (per master):

```
// Master N interface (APB slave side)
input  logic                    mN_apb_PSEL      // Select
input  logic                    mN_apb_PENABLE   // Enable
input  logic [ADDR_WIDTH-1:0]   mN_apb_PADDR     // Address
input  logic                    mN_apb_PWRITE    // Write enable
input  logic [DATA_WIDTH-1:0]   mN_apb_PWDATA    // Write data
input  logic [STRB_WIDTH-1:0]   mN_apb_PSTRB     // Write strobes
input  logic [2:0]              mN_apb_PPROT     // Protection
```

```
output logic [DATA_WIDTH-1:0] mN_apb_PRDATA    // Read data
output logic                  mN_apb_PSLVERR   // Error response
output logic                  mN_apb_PREADY    // Ready
```

Slave Interfaces (per slave):

```
// Slave N interface (APB master side)
output logic                  sN_apb_PSEL      // Select
output logic                  sN_apb_PENABLE   // Enable
output logic [ADDR_WIDTH-1:0] sN_apb_PADDR     // Address
output logic                  sN_apb_PWRITE    // Write enable
output logic [DATA_WIDTH-1:0] sN_apb_PWDATA    // Write data
output logic [STRB_WIDTH-1:0] sN_apb_PSTRB     // Write strobes
output logic [2:0]            sN_apb_PPROT     // Protection
input  logic [DATA_WIDTH-1:0] sN_apb_PRDATA    // Read data
input  logic                  sN_apb_PSLVERR   // Error response
input  logic                  sN_apb_PREADY    // Ready
```

2.5.3 Signal Descriptions

PSEL (Select): Indicates target of transfer (active high)

PENABLE (Enable): Indicates second cycle of APB transfer - PENABLE=0 → SETUP phase - PENABLE=1 → ACCESS phase

PADDR (Address): Byte address of transfer

PWRITE (Write): Direction of transfer - PWRITE=1 → Write - PWRITE=0 → Read

PWDATA (Write Data): Write data, valid when PWRITE=1

PSTRB (Strobes): Byte lane enables for write data - PSTRB[n]=1 → PWDATA[(n*8)+7:(n*8)] is valid

PPROT (Protection): Access protection attributes - PPROT[0]: 0=Normal, 1=Privileged - PPROT[1]: 0=Secure, 1=Non-secure - PPROT[2]: 0=Data, 1=Instruction

PRDATA (Read Data): Read data, valid when PREADY=1 and PWRITE=0

PSLVERR (Error): Transfer error indicator - PSLVERR=0 → OKAY - PSLVERR=1 → ERROR

PREADY (Ready): Slave ready indicator - PREADY=0 → Extend transfer - PREADY=1 → Complete transfer

---

2.6 Address Mapping

2.6.1 Default Address Map

Slaves are assigned sequential 64KB address regions starting from BASE_ADDR:

Slave	Address Range	Size	Calculation
Slave 0	[BASE_ADDR, BASE_ADDR + 0xFFFF]	64 KB	BASE_ADDR + 0*0x10000
Slave 1	[BASE_ADDR + 0x10000, BASE_ADDR + 0x1FFFF]	64 KB	BASE_ADDR + 1*0x10000
Slave 2	[BASE_ADDR + 0x20000, BASE_ADDR + 0x2FFFF]	64 KB	BASE_ADDR + 2*0x10000
Slave 3	[BASE_ADDR + 0x30000, BASE_ADDR + 0x3FFFF]	64 KB	BASE_ADDR + 3*0x10000
...	...	...	...
Slave N	[BASE_ADDR + N*0x10000, *BASE_ADDR* + *(N+1)*0x10000 - 1]	64 KB	BASE_ADDR + N*0x10000

2.6.2 Example: Default Configuration

With `BASE_ADDR = 0x1000_0000` and 4 slaves:

Address Range	Region	Description
0x1000_0000 - 0x1000_FFFF	Slave 0	APB peripheral 0
0x1001_0000 - 0x1001_FFFF	Slave 1	APB peripheral 1
0x1002_0000 - 0x1002_FFFF	Slave 2	APB peripheral 2
0x1003_0000 - 0x1003_FFFF	Slave 3	APB peripheral 3

2.6.3 Custom Address Map

For custom address mapping, modify the `BASE_ADDR` parameter:

```
apb_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h4000_0000)  // Custom base address
) u_xbar (
    // ...
);
```

Results in: - Slave 0: 0x4000_0000 - 0x4000_FFFF - Slave 1: 0x4001_0000 - 0x4001_FFFF - Slave 2: 0x4002_0000 - 0x4002_FFFF - Slave 3: 0x4003_0000 - 0x4003_FFFF

2.6.4 Alignment Requirement

The generated decoders do not compute the slave index arithmetically – they slice it out of `PADDR` directly. For the 4-slave variants:

```
addr_in_range = (cmd_paddr >= BASE_ADDR) &&
                (cmd_paddr <  (BASE_ADDR + 32'h0004_0000));
slave_sel     = cmd_paddr[17:16];
```

`BASE_ADDR` must therefore be aligned to the **total** decoded window (`num_slaves × 64 KB`; 256 KB for the 4-slave variants). An unaligned `BASE_ADDR` makes the range check and the index slice disagree, silently routing transfers to the wrong slave. The generator does not check this.

2.6.5 Decode Errors: Unmapped Addresses Stall

There is no decode-error response. The generated crossbars do not implement a default slave, and the following statements – which appeared in earlier revisions of this document – are **not** true of the RTL:

- ~~`PSLVERR = 1` on unmapped access~~
- ~~`PRDATA = 0xDEADBEEF` debug pattern~~
- ~~`PREADY = 1` immediate response~~

The string `DEADBEEF` does not appear anywhere in the crossbar RTL.

What actually happens: when `addr_in_range` is low, no `sN_cmd_valid` is asserted and `cmd_ready` back to the master's `apb_slave` stays low:

```
always_comb begin
    m0_cmd_ready = 1'b0;
    if (m0_cmd_valid && m0_addr_in_range) begin
        case (m0_slave_sel)
            2'd0: m0_cmd_ready = s0_cmd_ready;
            // ...
        endcase
    end
end
```

The command is never accepted, no response is ever produced, and `PREADY` is never asserted. **The APB transfer hangs indefinitely.** There is no timeout.

Integration requirement: do not expose an unmapped address range to a master through these crossbars. Either constrain the master's address map to the decoded window, or place an address filter with a default error slave upstream. If a master can be pointed at an arbitrary address (a CPU, a debugger), an errant access will wedge the bus until reset.

This is tracked as a limitation, not a bug in the current tests: the CocoTB testbenches exercise the mapped ranges only.

2.7 Arbitration

2.7.1 Round-Robin Arbitration

Each slave has independent round-robin arbitration when multiple masters request simultaneously.

Priority Rotation:

```
Initial:  M0 > M1 > M2 > M3
After M0: M1 > M2 > M3 > M0
After M1: M2 > M3 > M0 > M1
After M2: M3 > M0 > M1 > M2
...
```

2.7.2 Grant Persistence

Once a master is granted access to a slave: 1. Grant remains active during entire APB transaction (SETUP + ACCESS phases) 2. Response routing locked to requesting master 3. Grant released only after PREADY=1

This ensures: - No response data corruption - Atomic transactions - Predictable behavior

2.7.3 Arbitration Example

Scenario: Masters M0 and M1 both request Slave S0

```
Time 0: M0 requests S0 → M0 granted (highest priority)
Time 1: M0 SETUP phase
Time 2: M0 ACCESS phase, PREADY=0 (slave busy)
Time 3: M0 ACCESS phase, PREADY=1 (complete)
Time 4: M1 requests S0 → M1 granted (now highest priority)
Time 5: M1 SETUP phase
Time 6: M1 ACCESS phase, PREADY=1 (complete)
Time 7: M0 requests S0 → M0 granted (priority rotated back)
```

2.7.4 Arbitration Latency

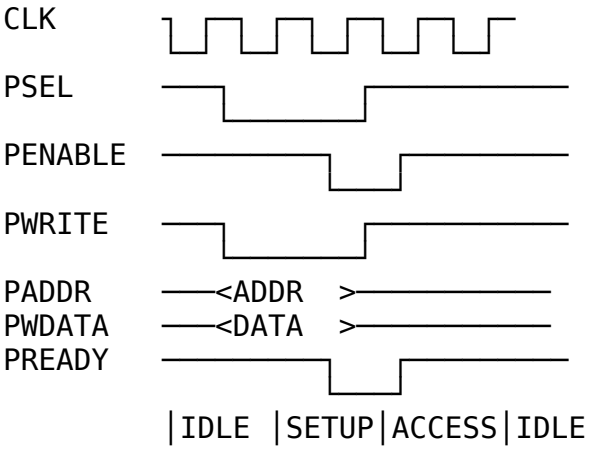
This table describes only the **arbitration** contribution. It is additive to the protocol-conversion cost described under Latency; it is not the end-to-end transfer latency.

Scenario	Arbitration cost	Description
No contention	0 cycles	No arbiter instantiated

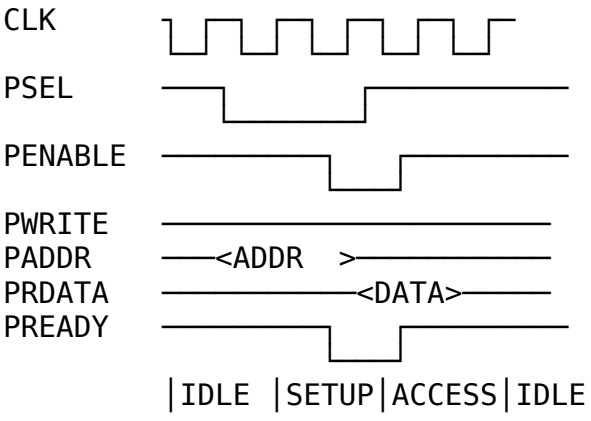
Scenario	Arbitration cost	Description
(single master)		for 1-master variants
Contention, slave idle	1 cycle	Grant registered before the request propagates
Contention, slave busy	1 cycle + remaining transaction	`WAIT_GNT_ACK=1` holds the grant until the current transfer completes

2.8 Timing Diagrams

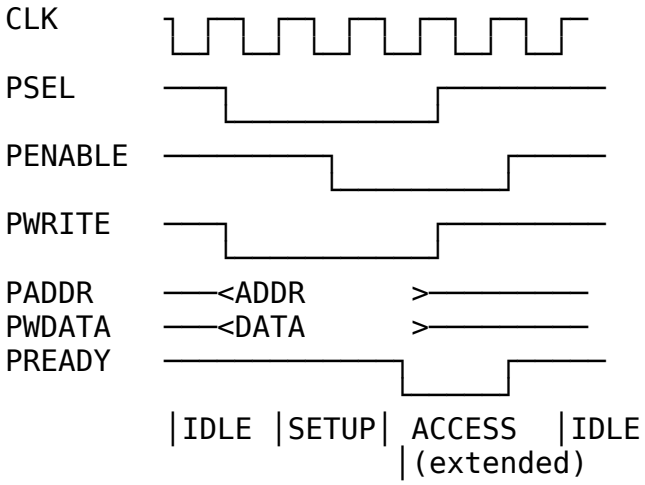
2.8.1 Basic Write Transaction (No Wait States)



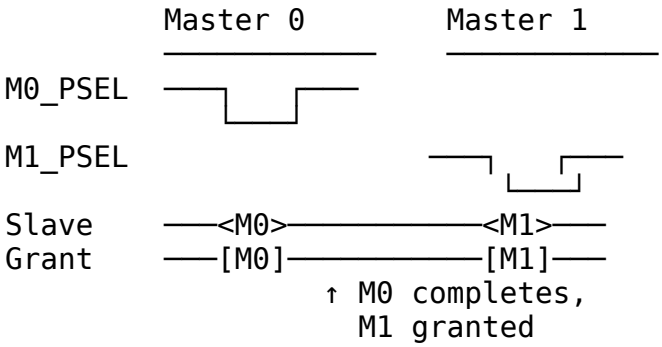
2.8.2 Basic Read Transaction (No Wait States)



2.8.3 Write with Wait States



2.8.4 Arbitration with Contention



2.9 Integration Examples

2.9.1 Example 1: Simple 1-to-4 Crossbar

```
module my_apb_subsystem (
    input  logic        pclk,
    input  logic        presetn,
    // Master APB interface
    input  logic        m_psel,
    input  logic        m_penable,
    input  logic [31:0] m_paddr,
    input  logic        m_pwrite,
    input  logic [31:0] m_pwdata,
    input  logic [3:0]  m_pstrb,
    output logic [31:0] m_prdata,
    output logic        m_pslverr,
    output logic        m_pready
```

```
);

    // Slave interfaces
    logic        s0_psel,   s1_psel,   s2_psel,   s3_psel;
    logic        s0_penable, s1_penable, s2_penable, s3_penable;
    logic [31:0] s0_paddr,  s1_paddr,  s2_paddr,  s3_paddr;
    logic        s0_pwrite, s1_pwrite, s2_pwrite, s3_pwrite;
    logic [31:0] s0_pwdata, s1_pwdata, s2_pwdata, s3_pwdata;
    logic [3:0]  s0_pstrb,  s1_pstrb,  s2_pstrb,  s3_pstrb;
    logic [31:0] s0_prdata, s1_prdata, s2_prdata, s3_prdata;
    logic        s0_pslverr, s1_pslverr, s2_pslverr, s3_pslverr;
    logic        s0_pready, s1_pready, s2_pready, s3_pready;

    // Instantiate crossbar
    apb_xbar_1to4 #(
        .ADDR_WIDTH (32),
        .DATA_WIDTH (32),
        .BASE_ADDR  (32'h1000_0000)
    ) u_xbar (
        .pclk       (pclk),
        .presetn    (presetn),
        // Master
        .m0_apb_PSEL    (m_psel),
        .m0_apb_PENABLE (m_penable),
        .m0_apb_PADDR   (m_paddr),
        .m0_apb_PWRITE  (m_pwrite),
        .m0_apb_PWDATA  (m_pwdata),
        .m0_apb_PSTRB   (m_pstrb),
        .m0_apb_PPROT   (3'b000),
        .m0_apb_PRDATA  (m_prdata),
        .m0_apb_PSLVERR (m_pslverr),
        .m0_apb_PREADY  (m_pready),
        // Slaves (connect to your peripherals)
        .s0_apb_PSEL    (s0_psel),
        .s0_apb_PENABLE (s0_penable),
        .s0_apb_PADDR   (s0_paddr),
        .s0_apb_PWRITE  (s0_pwrite),
        .s0_apb_PWDATA  (s0_pwdata),
        .s0_apb_PSTRB   (s0_pstrb),
        .s0_apb_PPROT   (),
        .s0_apb_PRDATA  (s0_prdata),
        .s0_apb_PSLVERR (s0_pslverr),
        .s0_apb_PREADY  (s0_pready),
        // Repeat for s1, s2, s3...
    );

    // Connect slaves to peripherals
    uart u_uart (
```

```
        .pclk       (pclk),
        .presetn    (presetn),
        .psel       (s0_psel),
        .penable    (s0_penable),
        .paddr      (s0_paddr[7:0]),
        .pwrite     (s0_pwrite),
        .pwdata     (s0_pwdata),
        .prdata     (s0_prdata),
        .pslverr    (s0_pslverr),
        .pready     (s0_pready)
    );

    gpio u_gpio (
        .pclk       (pclk),
        .presetn    (presetn),
        .psel       (s1_psel),
        // ... (similar connections)
    );

    // ... more peripherals

endmodule
```

2.9.2 Example 2: Multi-Master System (2-to-4)

```
module multi_master_apb_system (
    input  logic        pclk,
    input  logic        presetn,
    // CPU APB master
    input  logic        cpu_psel,
    input  logic [31:0] cpu_paddr,
    // ... (rest of CPU APB signals)
    // DMA APB master
    input  logic        dma_psel,
    input  logic [31:0] dma_paddr,
    // ... (rest of DMA APB signals)
);

    apb_xbar_2to4 #(
        .ADDR_WIDTH (32),
        .DATA_WIDTH (32),
        .BASE_ADDR  (32'h4000_0000)
    ) u_xbar (
        .pclk       (pclk),
        .presetn    (presetn),
        // Master 0 (CPU)
        .m0_apb_PSEL    (cpu_psel),
        .m0_apb_PENABLE (cpu_penable),
```

```
        .m0_apb_PADDR   (cpu_paddr),
        // ... (rest of CPU connections)
        // Master 1 (DMA)
        .m1_apb_PSEL    (dma_psel),
        .m1_apb_PENABLE (dma_penable),
        .m1_apb_PADDR   (dma_paddr),
        // ... (rest of DMA connections)
        // Slaves 0-3 (peripherals)
        // ... (connect to your peripherals)
    );

endmodule
```

2.9.3 Example 3: Custom Configuration (3x8 Crossbar)

```
# Generate custom 3-master, 8-slave crossbar
cd projects/components/apb_xbar/bin
python generate_xbars.py --masters 3 --slaves 8 --base-addr 0x80000000
```

Instantiate:

```
apb_xbar_3to8 #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .BASE_ADDR  (32'h8000_0000)  // Custom base
) u_custom_xbar (
    // 3 master interfaces (m0, m1, m2)
    // 8 slave interfaces (s0-s7)
);
```

2.10 Test Results

All crossbar modules have CocoTB testbenches in `projects/components/apb_xbar/dv/tests/`.

2.10.1 Test Coverage

Test	Description	Status
`test_apb_xbar_1to1`	Basic passthrough, protocol conversion	✓ PASS
`test_apb_xbar_2to1`	Arbitration, fairness, grant persistence	✓ PASS
`test_apb_xbar_1to4`	Address decode, multiple slaves	✓ PASS

Test	Description	Status
`test_apb_xbar_2to4`	Full crossbar, arbitration + decode	√ PASS

Each test drives a `*_wrap` wrapper (`rtl/wrappers/apb_xbar_*_wrap.sv`) rather than the bare crossbar.

2.10.2 Formal Verification

SymbiYosys proofs exist for all four generated variants plus `apb_xbar_thin` under `formal/apb_xbar/`, each with `prove` and `cover` tasks.

2.10.3 Test Scenarios

Each test includes: - √ Basic read/write transactions - √ Back-to-back transfers - √ Wait state handling - √ Error responses (`PSLVERR` propagated from a mapped slave) - √ Address decode verification (mapped ranges) - √ Arbitration fairness (multi-master) - √ Concurrent transactions (different slaves) - √ Reset behavior

Not covered:

- ✗ Unmapped-address access – would hang; see Decode Errors
- ✗ Unaligned `BASE_ADDR`

2.10.4 Running Tests

```
source env_python

# Run all APB crossbar tests
pytest projects/components/apb_xbar/dv/tests/ -v

# Run specific variant
pytest projects/components/apb_xbar/dv/tests/test_apb_xbar_2to4.py -v
```

Build artifacts and logs are preserved under `projects/components/apb_xbar/dv/tests/local_sim_build/` and `.../logs/`.

2.10.5 Test Results Summary

Measured 2026-07-19 with Verilator:

```
test_apb_xbar_1to1.py::test_apb_xbar_1to1   PASSED
test_apb_xbar_2to1.py::test_apb_xbar_2to1   PASSED
test_apb_xbar_1to4.py::test_apb_xbar_1to4   PASSED
test_apb_xbar_2to4.py::test_apb_xbar_2to4   PASSED  (350 transactions)

4 passed in 10.14s
```

2.11 Known Limitations

2.11.1 Current Limitations

1. **Fixed 64KB Slave Regions**
   - Each slave has 64KB address space
   - Cannot be changed without regenerating module
   - **Workaround**: Generate custom crossbar with modified generator
2. **No Outstanding Transaction Support**
   - APB is inherently non-pipelined
   - One transaction completes before next starts
   - **Impact**: Lower throughput than pipelined protocols (AXI)
3. **No Sparse Addressing**
   - Slaves must be in contiguous 64KB blocks
   - Cannot have gaps in address map
   - `BASE_ADDR` must be aligned to the total decoded window
   - **Workaround**: Leave unused slaves unconnected
4. **Round-Robin Only**
   - Fixed round-robin arbitration
   - No priority levels or quality-of-service
   - **Workaround**: use `apb_xbar_thin`, which has weighted round-robin, or an external priority arbiter ahead of the crossbar
5. **Unmapped Addresses Hang the Bus**
   - No default slave, no decode-error response, no timeout
   - An access outside the decoded window never receives `PREADY`
   - **Workaround**: constrain the master's address map, or place an address filter with an error slave upstream
   - See Decode Errors

2.11.2 Future Enhancements

- ☐ Configurable slave address regions
- ☐ Priority-based arbitration option
- ☐ Sparse address map support
- ☐ Performance counters

- ☐ Timeout detection

2.11.3 Known Issues

All generated crossbars pass their CocoTB and formal verification. The unmapped-address stall described above is a documented architectural limitation rather than a test failure – the testbenches only drive mapped ranges, so it is not caught by the current suite.

2.12 Related Documentation

Generator Tutorial: - `docs/markdown/GeneratorTutorial/apb_xbar_generator_tutorial.md` - How to use generator

APB Protocol: - ARM IHI 0024C – AMBA APB Protocol Specification, Version 2.0 (APB4) - APB module index - Protocol overview and signal descriptions

Building Blocks: - `rtl/amba/apb/apb_slave.sv` - APB → cmd/rsp conversion (apb_slave.md) - `rtl/amba/apb/apb_master.sv` - cmd/rsp → APB conversion (apb_master.md) - `rtl/amba/gaxi/gaxi_skid_buffer.sv` - Command and response buffering - `rtl/common/arbiter_round_robin.sv` - Per-slave arbitration (`WAIT_GNT_ACK=1`)

Generator: - `projects/components/apb_xbar/bin/apb_xbar_generator.py` - Crossbar generator - `projects/components/apb_xbar/bin/generate_xbars.py` - Convenience script - `projects/components/apb_xbar/README.md` - Quick reference

---

2.13 Revision History

Version	Date	Author	Changes
1.0	2025-10-14	RTL Design Sherpa	Initial comprehensive specification
1.1	2026-07-19	RTL Design Sherpa	Corrected module/test paths to the `apb_xbar` component area; APB5 claim reduced to APB4; removed unsubstantiated

Version	Date	Author	Changes
			latency, LUT, MHz and transaction-count figures; documented that unmapped addresses stall rather than returning `PSLVERR`/`0xDEADBEEF`; added `BASE_ADDR` alignment requirement

Maintained By: RTL Design Sherpa Project **License:** MIT **Contact:** See repository for support

---

2.14 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

3 apb_master

An Advanced Peripheral Bus (APB) master module that provides a high-performance interface for accessing APB slave devices with command/response buffering and full AMBA 4 APB (APB4) protocol compliance.

Protocol scope: APB4 only. For `PWAKEUP`, the `P*USER` sidebands, or optional parity, use `apb5_master` from `rtl/amba/apb5/` – see the APB5 book.

3.1 Overview

The `apb_master` module implements a complete APB master interface with integrated command and response buffers for improved system performance. It supports all APB4 features including write strobes, protection attributes, and error responses while providing a simple command/response interface for system integration.

3.2 Module Declaration

```
module apb_master #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int PROT_WIDTH      = 3,
    parameter int CMD_DEPTH       = 6,
    parameter int RSP_DEPTH       = 6,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Master Interface
    output logic              m_apb_PSEL,
    output logic              m_apb_PENABLE,
    output logic [AW-1:0]     m_apb_PADDR,
    output logic              m_apb_PWRITE,
    output logic [DW-1:0]     m_apb_PWDATA,
    output logic [SW-1:0]     m_apb_PSTRB,
    output logic [PW-1:0]     m_apb_PPROT,
    input  logic [DW-1:0]     m_apb_PRDATA,
    input  logic              m_apb_PSLVERR,
    input  logic              m_apb_PREADY,

    // Command Interface
    input  logic              cmd_valid,
    output logic              cmd_ready,
    input  logic              cmd_pwrite,
    input  logic [AW-1:0]     cmd_paddr,
    input  logic [DW-1:0]     cmd_pwdata,
    input  logic [SW-1:0]     cmd_pstrb,
    input  logic [PW-1:0]     cmd_pprot,

    // Response Interface
    output logic              rsp_valid,
    input  logic              rsp_ready,
    output logic [DW-1:0]     rsp_prdata,
```

```
    output logic                rsp_pslverr
);
```

3.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	APB protection signal width
CMD_DEPTH	int	6	Command buffer depth in **entries** (not a log2 exponent)
RSP_DEPTH	int	6	Response buffer depth in **entries** (not a log2 exponent)
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

CMD_DEPTH and RSP_DEPTH are literal entry counts. Both buffers are `gaxi_skid_buffer` instances, which store their payload in an unpacked array of `DEPTH` slots. The default of 6 means six entries, not sixty-four. Supported values are `{2, 4, 6, 8}`.

3.4 Ports

3.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

3.4.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal

Port	Width	Direction	Description
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	APB write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	APB write data
m_apb_PSTRB	STRB_WIDTH	Output	APB write strobes
m_apb_PPROT	PROT_WIDTH	Output	APB protection attributes
m_apb_PRDATA	DATA_WIDTH	Input	APB read data
m_apb_PSLVERR	1	Input	APB slave error
m_apb_PREADY	1	Input	APB ready

3.4.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection

Port	Width	Direction	Description
			attributes

3.4.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data
rsp_pslverr	1	Output	Response error status

3.5 Functionality

3.5.1 APB Protocol Implementation

The module implements the complete APB protocol state machine:

1. **IDLE**: No transaction active, waiting for commands
2. **SETUP**: PSEL asserted, transaction parameters set up
3. **ACCESS**: PENABLE asserted, waiting for PREADY from slave

3.5.2 Command Processing Flow

```
Command Interface → Command FIFO → APB State Machine → APB Interface
                                                              ↓
Response Interface ← Response FIFO ← APB Response Processing
```

3.5.3 Key Features

- **Buffered Operation**: Command and response buffers prevent blocking
- **APB4 Compliance**: Full protocol support including PSTRB and PPROT
- **Error Handling**: Proper PSLVERR propagation and handling
- **Flow Control**: Ready/valid handshaking on all interfaces

3.6 Timing Characteristics

3.6.1 APB Transaction Timing

Characteristic	Value	Description
Setup Phase	1 clock cycle	PSEL assertion
Access Phase	≥1 clock cycle	PENABLE assertion until PREADY
Total Latency	2+ clock cycles	Minimum transaction time
FIFO Latency	1 clock cycle	Command to APB delay
Response Latency	1 clock cycle	APB to response delay

3.6.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent, not characterized in this repository
Command Buffering	6 commands	With default `CMD_DEPTH=6`
Response Buffering	6 responses	With default `RSP_DEPTH=6`
Outstanding APB transfers	1	APB is non-pipelined

On throughput: APB is non-pipelined – the FSM issues one transfer at a time and cannot start the next until `PREADY` for the current one. Buffering lets the command producer run ahead and lets responses drain lazily; it does not overlap bus transfers. Peak throughput is therefore `DATA_WIDTH / (transfer cycles × pclk period)`, with a minimum of two cycles (SETUP + one ACCESS cycle) per transfer against a zero-wait-state slave.

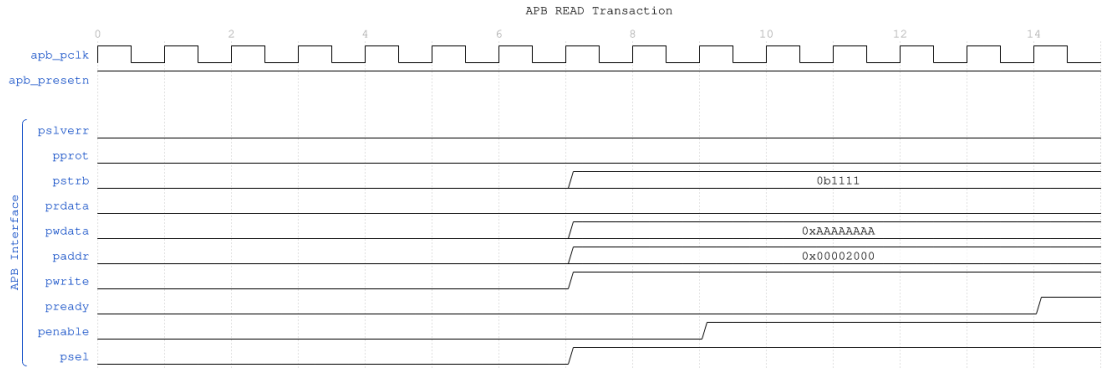
3.7 Waveforms

The following timing diagrams show comprehensive APB master behavior across 6 scenarios.

Known diagram defect: the WaveDrom generator stamps a fixed header string, `APB READ Transaction`, onto every scenario image regardless of direction. The signal traces themselves are correct – read `PWRITE` in the trace, not the header, to determine transaction direction.

3.7.1 Scenario 1: Basic Write Transaction

Shows APB write with command FIFO and response handling:

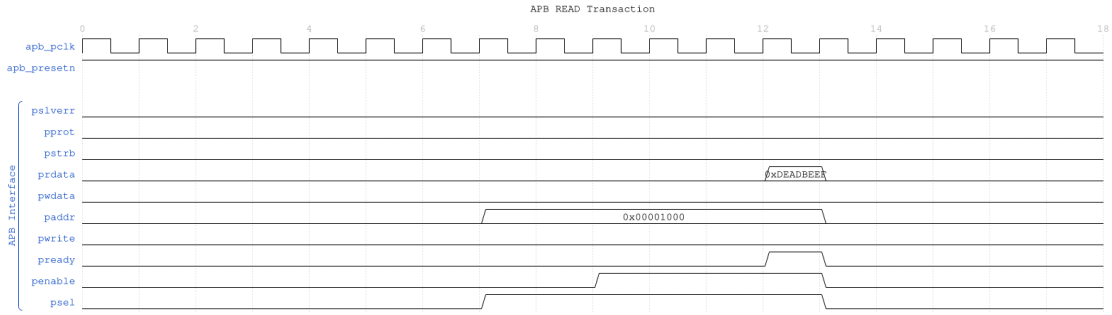


APB Write

WaveJSON: apb_write_sequence_001.json

Key Observations: - Command accepted into CMD FIFO - APB SETUP phase (PSEL=1) - APB ACCESS phase (PENABLE=1) - Response captured in RSP FIFO

3.7.2 Scenario 2: Basic Read Transaction

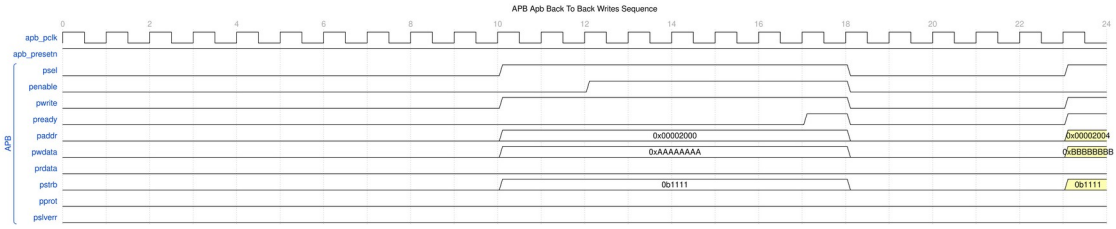


APB Read

WaveJSON: apb_read_sequence_001.json

Key Observations: - Read command with PWRITE=0 - APB protocol phases - Read data returned via response interface

3.7.3 Scenario 3: Back-to-Back Writes

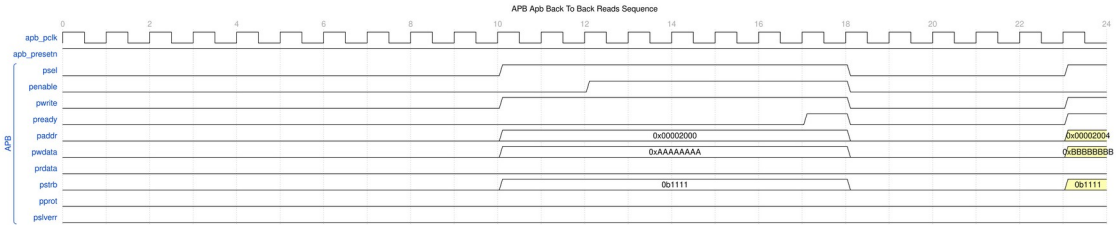


B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

Key Observations: - Multiple commands queued in CMD FIFO - Sequential APB transactions - Pipelined command/response flow

3.7.4 Scenario 4: Back-to-Back Reads

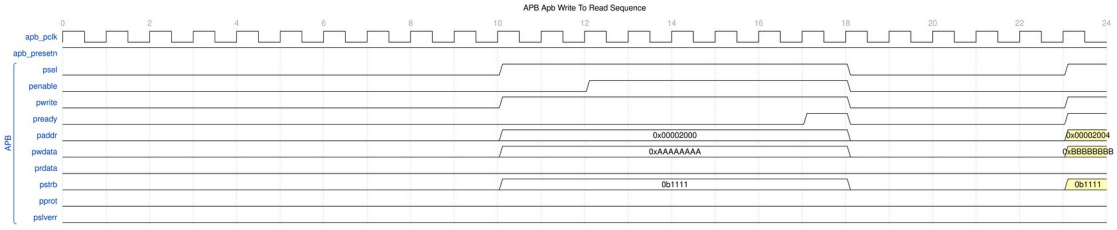


B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

Key Observations: - Multiple read commands queued - Response FIFO accumulates read data - Burst read performance

3.7.5 Scenario 5: Write-to-Read Transition

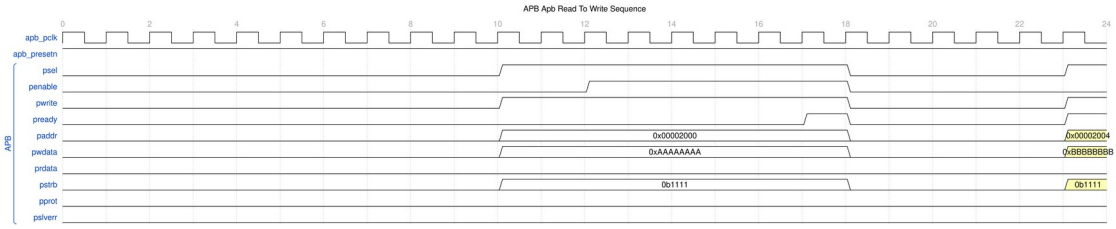


Write-to-Read

WaveJSON: apb_write_to_read_001.json

Key Observations: - Transition between write and read operations - No dead cycles between transaction types

3.7.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

Key Observations: - Seamless transition from read to write - Protocol state machine efficiency

---

3.8 Usage Examples

3.8.1 Basic APB Master Configuration

```
apb_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .CMD_DEPTH(4),      // 16-entry command FIFO
    .RSP_DEPTH(4)       // 16-entry response FIFO
) u_apb_master (
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // APB interface to slaves
    .m_apb_PSEL     (apb_psel),
    .m_apb_PENABLE  (apb_penable),
    .m_apb_PADDR    (apb_paddr),
    .m_apb_PWRITE   (apb_pwrite),
    .m_apb_PWDATA   (apb_pwdata),
    .m_apb_PSTRB    (apb_pstrb),
    .m_apb_PPROT    (apb_pprot),
    .m_apb_PRDATA   (apb_prdata),
    .m_apb_PSLVERR  (apb_pslverr),
    .m_apb_PREADY   (apb_pready),

    // Command interface from CPU/DMA
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_write),
    .cmd_paddr      (cmd_addr),
    .cmd_pwdata     (cmd_wdata),
```

```
    .cmd_pstrb      (cmd_strb),
    .cmd_pprot      (3'b000),   // Normal access

    // Response interface to CPU/DMA
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_rdata),
    .rsp_pslverr    (rsp_error)
);
```

3.8.2 Register Write Sequence

```
// Write to control register
always_ff @(posedge clk) begin
    if (write_control_reg) begin
        cmd_valid  <= 1'b1;
        cmd_pwrite <= 1'b1;
        cmd_paddr  <= 32'h1000_0000;    // Control register address
        cmd_pwdata <= control_value;
        cmd_pstrb  <= 4'hF;             // All bytes valid
        cmd_pprot  <= 3'b000;           // Normal access
    end else if (cmd_ready) begin
        cmd_valid  <= 1'b0;
    end
end

// No response needed for writes (unless checking for errors)
assign rsp_ready = 1'b1;
```

3.8.3 Register Read Sequence

```
// Read from status register
always_ff @(posedge clk) begin
    if (read_status_reg) begin
        cmd_valid  <= 1'b1;
        cmd_pwrite <= 1'b0;             // Read operation
        cmd_paddr  <= 32'h1000_0004;    // Status register address
        cmd_pwdata <= 32'h0;            // Don't care for reads
        cmd_pstrb  <= 4'hF;             // All bytes
        cmd_pprot  <= 3'b000;           // Normal access
    end else if (cmd_ready) begin
        cmd_valid  <= 1'b0;
    end
end

// Handle read response
always_ff @(posedge clk) begin
    if (rsp_valid && rsp_ready) begin
        if (!rsp_pslverr) begin
```

```
            status_value <= rsp_prdata;
            read_complete <= 1'b1;
        end else begin
            read_error <= 1'b1;
        end
    end
end

assign rsp_ready = 1'b1;  // Always ready to accept responses
```

3.8.4 Burst Operation Example

```
// Multiple register access sequence
parameter int NUM_REGS = 8;
logic [31:0] reg_addresses [NUM_REGS] = '{
    32'h1000_0000, 32'h1000_0004, 32'h1000_0008, 32'h1000_000C,
    32'h1000_0010, 32'h1000_0014, 32'h1000_0018, 32'h1000_001C
};
logic [31:0] reg_data [NUM_REGS];
int reg_index;

// Issue multiple commands
always_ff @(posedge clk) begin
    if (start_burst && reg_index < NUM_REGS) begin
        if (cmd_ready || !cmd_valid) begin
            cmd_valid  <= 1'b1;
            cmd_pwrite <= 1'b1;
            cmd_paddr  <= reg_addresses[reg_index];
            cmd_pwdata <= reg_data[reg_index];
            cmd_pstrb  <= 4'hF;
            cmd_pprot  <= 3'b000;
            reg_index  <= reg_index + 1;
        end
    end else begin
        cmd_valid <= 1'b0;
    end
end
```

3.9 Integration with APB Interconnect

3.9.1 APB Crossbar Integration

```
// APB system with crossbar
module apb_system (
    input logic clk, resetn,
    // CPU interface
    input logic [31:0] cpu_addr,
    input logic [31:0] cpu_wdata,
```

```
    input logic         cpu_write,
    input logic         cpu_valid,
    output logic        cpu_ready,
    output logic [31:0] cpu_rdata,
    output logic         cpu_error
);

    // APB master
    apb_master u_apb_master (
        .pclk(clk),
        .presetn(resetn),
        .m_apb_*(apb_m_*),
        // CPU command interface
        .cmd_valid(cpu_valid),
        .cmd_ready(cpu_ready),
        .cmd_pwrite(cpu_write),
        .cmd_paddr(cpu_addr),
        .cmd_pwdata(cpu_wdata),
        .cmd_pstrb(4'hF),
        .cmd_pprot(3'b000),
        // CPU response interface
        .rsp_valid(rsp_valid),
        .rsp_ready(1'b1),
        .rsp_prdata(cpu_rdata),
        .rsp_pslverr(cpu_error)
    );

    // APB crossbar to multiple slaves.
    // Fixed-configuration generated crossbar: 1 master, 4 slaves.
    // Its master-side ports are m0_apb_* (inputs from this master)
and its
    // slave-side ports are sN_apb_* (outputs to the peripherals).
    apb_xbar_1to4 #(
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32),
        .BASE_ADDR (32'h1000_0000)
    ) u_apb_xbar (
        .pclk(clk),
        .presetn(resetn),
        // Master side: driven by the apb_master above
        .m0_apb_*(apb_m_*),
        // Slave side: to the peripherals
        .s0_apb_*(slave0_apb_*),
        .s1_apb_*(slave1_apb_*),
        .s2_apb_*(slave2_apb_*),
        .s3_apb_*(slave3_apb_*)
    );
```

```
endmodule
```

3.10 Advanced Features

3.10.1 Protection Attributes (PPROT)

The module passes APB protection attributes through unmodified. Per the AMBA APB specification each bit is independent:

Bit	0	1
PPROT[0]	Normal access	Privileged access
PPROT[1]	Secure access	Non-secure access
PPROT[2]	Data access	Instruction access

Note the polarity of PPROT[1]: **0 means secure**, 1 means non-secure. The full encoding follows from the per-bit table:

PPROT[2:0]	Description
000	Normal, Secure, Data
001	Privileged, Secure, Data
010	Normal, Non-secure, Data
011	Privileged, Non-secure, Data
100	Normal, Secure, Instruction
101	Privileged, Secure, Instruction
110	Normal, Non-secure, Instruction
111	Privileged, Non-secure, Instruction

3.10.2 Write Strobe Support

Fine-grained byte-level write control:

```
// Example: Write only upper 16 bits of 32-bit register
cmd_pwdata <= {new_upper_data, 16'h0000};
cmd_pstrb  <= 4'b1100;  // Only upper 2 bytes
```

3.10.3 Error Handling

Comprehensive error handling and reporting:

```
always_ff @(posedge clk) begin
    if (rsp_valid && rsp_ready) begin
        if (rsp_pslverr) begin
            // Log error details
            error_count <= error_count + 1;
            error_addr  <= last_cmd_addr;
            error_type  <= last_cmd_write ? ERR_WRITE : ERR_READ;
        end
    end
end
```

3.11 Performance Optimization

3.11.1 FIFO Depth Selection

Depths are entry counts and must be one of `{2, 4, 6, 8}`. Because APB is non-pipelined, deeper buffers do not increase bus throughput – they only let the command producer run further ahead of the bus and absorb bursty response draining.

Application	CMD_DEPTH	RSP_DEPTH	Rationale
CPU Access	2	2	Low latency, one access at a time
DMA / block transfer	6	6	Producer runs ahead of the bus
Bridged (AXI4-to-APB)	8	8	Upstream pipelines commands well ahead of responses

3.11.2 Clock Gating

For power-sensitive applications, use `apb_master_cg`. It wraps `apb_master` in `amba_clock_gate_ctrl` and adds runtime configuration inputs rather than compile-time enables:

```
apb_master_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CG_IDLE_COUNT_WIDTH (4)   // width of the idle countdown
) u_apb_master_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),
```

```
    // Clock gating control (runtime inputs, not parameters)
    .cfg_cg_enable     (cg_enable),      // 1 = allow gating
    .cfg_cg_idle_count (4'd8),           // idle cycles before gating

    // ... all other ports identical to apb_master ...

    // Status
    .apb_clock_gating  (apb_cg_active)   // 1 while the clock is gated
);
```

See apb_master_cg.md for the full interface.

3.12 Synthesis Considerations

3.12.1 Area Optimization

- Reduce FIFO depths for area-constrained designs
- Use synchronous reset for smaller area
- Consider shared resources for multiple APB masters

3.12.2 Timing Optimization

- Register all outputs for timing closure
- Use appropriate FIFO depths to break critical paths
- Consider skid buffers for high-frequency operation

3.12.3 Power Optimization

- Use clock gating variant when available
- Implement power gating for inactive masters
- Use appropriate FIFO depths to minimize switching

3.13 Verification Notes

3.13.1 Protocol Compliance

- Verify APB setup and access phases
- Check PSEL and PENABLE timing
- Validate PREADY handling and wait states

3.13.2 Interface Verification

- Test command/response FIFO overflow/underflow
- Verify backpressure handling

- Check error propagation and handling

3.13.3 Performance Verification

- Measure sustained throughput
- Verify FIFO efficiency
- Check latency under various loads

3.14 Related Modules

- **apb_master_cg**: Clock-gated version for power optimization
- **apb_master_stub**: Packed-interface variant wrapping this module, adds first/last framing
- **apb_slave**: APB slave implementation
- **apb5_master**: APB5 equivalent with `PWAKEUP`, `P*USER` and optional parity
- **apb_xbar_1to4** / **apb_xbar_2to4**: Generated APB crossbars for multi-slave systems
- **apb_monitor**: APB protocol monitor and analyzer

The `apb_master` module provides a complete, high-performance solution for APB master functionality with advanced buffering, full protocol compliance, and comprehensive system integration capabilities.

3.15 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

4 APB Master Interface (Clock-Gated)

Module: `apb_master_cg.sv` **Base Module:** apb_master **Location:** `rtl/amba/apb/` **Status:** ✓ Production Ready

---

4.1 Quick Reference

This is the **clock-gated variant** of apb_master.

For the clock-gating architecture and the underlying gate cell, see:

→ **Clock-Gated Variants Guide** and → **amba_clock_gate_ctrl**

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

4.2 Summary

The `apb_master_cg` module adds power optimization to `apb_master` through activity-based clock gating. It is a thin wrapper: an `amba_clock_gate_ctrl` instance produces `gated_pclk`, which feeds an otherwise unmodified `apb_master`.

- √ **Same Functionality:** 100% equivalent to base module
- √ **Runtime Control:** Gating is enabled and tuned by input signals, not parameters
- √ **Observable:** `apb_clock_gating` output reports when the clock is gated
- √ **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

---

4.3 Additional Parameters

In addition to all apb_master parameters:

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter; bounds the maximum programmable idle threshold

There is no `ENABLE_CLOCK_GATING` parameter and no per-domain `CG_GATE_*` parameters on this module – gating is controlled at runtime through the configuration inputs below.

4.4 Additional Ports

In addition to all apb_master ports:

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Global clock-gate enable. 0 = never gate (identical to

Port	Width	Direction	Description
			base module)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating the clock
apb_clock_gating	1	Output	Asserted while the internal clock is gated

4.4.1 Wake-Up Condition

The wrapper holds the clock ungated while any of the following is true, sampled one cycle earlier into an internal `r_wakeup` register:

```
cmd_valid || rsp_valid || m_apb_PSEL || m_apb_PENABLE
```

That is: a pending command, a pending response, or an APB transfer in progress. Gating engages `cfg_cg_idle_count + 1` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after all four terms go low, because APB adds two register stages ahead of the ICG enable.

4.4.2 Wake-Up Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This wrapper registers activity into its own `r_wakeup` flop and then hands it to `amba_clock_gate_ctrl`, which registers it again, so APB is a **two-stage** family. The first gated-clock rising edge available to the wrapped `apb_master` therefore arrives **3 cycles** after activity asserts.

4.5 Quick Usage

```
apb_master_cg #(
    // Base module parameters (see apb_master.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CMD_DEPTH           (6),
    .RSP_DEPTH           (6),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    // Clock gating control
```

```
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),
    .apb_clock_gating  (apb_cg_active),

    // ... all other ports same as apb_master
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging waveforms. A gated clock makes the internal state appear frozen and is easy to misread as a hang.

4.6 Documentation

- **Base Module Functionality:** apb_master.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

4.7 Navigation

- **← Back to APB Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

5 apb_master_stub

A lightweight APB master stub module that provides packed command/response interfaces for simplified testbench integration and system-level testing.

5.1 Overview

The `apb_master_stub` module acts as a simple APB master that accepts packed command packets and returns packed response packets. It's designed for testbench use where a simple APB master is needed without the full functionality of the standard `apb_master` module. The packed interface simplifies integration with test drivers and verification components.

5.2 Module Declaration

```
module apb_master_stub #(
    parameter int CMD_DEPTH         = 6,
    parameter int RSP_DEPTH         = 6,
    parameter int DATA_WIDTH        = 32,
    parameter int ADDR_WIDTH        = 32,
    parameter int STRB_WIDTH        = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 3 + 1 + 1 + 1,
                                        // addr, data, strb, prot,
pwrite, first, last
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1 + 1 + 1, // data,
pslverr, first, last
    // Short Parameters
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
    // Clock and Reset
    input  logic                          pclk,
    input  logic                          presetn,

    // APB Master Interface
    output logic                          m_apb_PSEL,
    output logic                          m_apb_PENABLE,
    output logic [ADDR_WIDTH-1:0]         m_apb_PADDR,
    output logic                          m_apb_PWRITE,
    output logic [DATA_WIDTH-1:0]         m_apb_PWDATA,
    output logic [STRB_WIDTH-1:0]         m_apb_PSTRB,
    output logic [2:0]                    m_apb_PPROT,
    input  logic [DATA_WIDTH-1:0]         m_apb_PRDATA,
    input  logic                          m_apb_PSLVERR,
    input  logic                          m_apb_PREADY,

    // Command Packet Interface (packed)
    input  logic                          cmd_valid,
    output logic                          cmd_ready,
    input  logic [CMD_PACKET_WIDTH-1:0]   cmd_data,

    // Response Packet Interface (packed)
    output logic                          rsp_valid,
    input  logic                          rsp_ready,
    output logic [RESP_PACKET_WIDTH-1:0] rsp_data
);
```

5.3 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command buffer depth in **entries** (not a log2 exponent)
RSP_DEPTH	int	6	Response buffer depth in **entries** (not a log2 exponent)
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

5.4 Ports

5.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

5.4.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	Peripheral select
m_apb_PENABLE	1	Output	Enable signal
m_apb_PADDR	AW	Output	Address bus
m_apb_PWRITE	1	Output	Write/read (1=write,

Port	Width	Direction	Description
			0=read)
m_apb_PWDATA	DW	Output	Write data
m_apb_PSTRB	SW	Output	Write strobe
m_apb_PPROT	3	Output	Protection attributes
m_apb_PRDATA	DW	Input	Read data
m_apb_PSLVERR	1	Input	Slave error
m_apb_PREADY	1	Input	Ready signal

5.4.3 Packed Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready for command packet
cmd_data	CPW	Input	Packed command (addr, data, strb, prot, pwrite, first, last)

5.4.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready for response packet
rsp_data	RPW	Output	Packed response (data, pslverr, first, last)

5.5 Functional Description

5.5.1 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `pwdata` (DW bits): Write data - `paddr` (AW bits): Address

Response Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

5.5.2 First/Last Framing

`first` and `last` exist so that AXI4 burst framing survives the trip through an APB bridge. `apb_master` does not carry them through its own command-to-response pipeline, so the stub keeps a small side FIFO (`gaxi_fifo_sync`, depth `CMD_DEPTH`): `{last, first}` is enqueued on every command-side handshake and dequeued on every response-side handshake.

This pairing is essential when the upstream pipelines commands ahead of responses. Tapping `first`/`last` combinationally from the live `cmd_data` input instead – as an earlier revision did – pairs command N's response with command N+1's framing bits as soon as the internal command FIFO accepts N+1. In the AXI4-to-APB bridge that manifested as `axi4_to_apb_convert` never seeing `first=1` again in `RSP_IDLE`, hanging the FSM and surfacing as a timeout on the master AXI4 R channel.

Because the side FIFO mirrors the depth of the `apb_master` command buffer, it never backpressures the command path more than that buffer already does.

5.5.3 Packet Format Is Not Symmetric With `apb_slave_stub`

`apb_slave_stub` omits `first` and `last` in both directions and is therefore two bits narrower per packet. The two stubs face each other across the APB bus, never packed-side to packed-side, so this asymmetry is intentional and causes no integration problem – but the packed buses must not be wired together directly. See apb_slave_stub.md for the side-by-side comparison.

5.5.4 Operation

1. Test driver presents packed command on `cmd_data` with `cmd_valid=1`
2. Stub accepts when `cmd_ready=1`
3. Stub unpacks command, enqueues `{last, first}`, and drives APB protocol
4. On APB completion, stub packs the response with the dequeued `{last, first}` and asserts `rsp_valid=1`
5. Test driver reads response when `rsp_ready=1`

5.6 Usage Example

```
// Instantiate APB master stub
apb_master_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb_master_stub (
```

```
    .pclk(clk),
    .presetn(rst_n),
    // APB interface to slave/interconnect
    .m_apb_PSEL(apb_psel),
    .m_apb_PENABLE(apb_penable),
    .m_apb_PADDR(apb_paddr),
    .m_apb_PWRITE(apb_pwrite),
    .m_apb_PWDATA(apb_pwdata),
    .m_apb_PSTRB(apb_pstrb),
    .m_apb_PPROT(apb_pprot),
    .m_apb_PRDATA(apb_prdata),
    .m_apb_PSLVERR(apb_pslverr),
    .m_apb_PREADY(apb_pready),
    // Packed interface to test driver
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Send write command (in testbench)
// Pack command: addr=0x1000, data=0xDEADBEEF, write=1
assign test_cmd_data = {1'b1, 1'b1, 1'b1, 3'b000, 4'hF, 32'hDEADBEEF,
16'h1000};
assign test_cmd_valid = 1'b1;
```

5.7 Design Notes

5.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB traffic generation - Verification component integration

For production use, consider the full-featured `apb_master` module.

5.7.2 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for parameterized test sequences

5.8 Related Modules

- `apb_master.sv` - Full-featured APB master with independent interfaces

- `apb_slave_stub.sv` - Companion APB slave stub
- `apb_monitor.sv` - APB transaction monitoring

5.9 References

- **APB Protocol**: ARM IHI 0024C – AMBA APB Protocol Specification, Version 2.0 (APB4)
- **Full APB Master**: apb_master.md
- **APB Slave Stub**: apb_slave_stub.md
- **APB5 Equivalent**: apb5_master_stub.md

5.10 Navigation

- **← Back to APB Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

6 apb_slave

An Advanced Peripheral Bus (APB) slave module that provides a complete AMBA 4 APB (APB4) compliant interface with buffered command/response processing for high-performance peripheral integration.

Protocol scope: APB4 only. For `PWAKEUP`, the `P*USER` sidebands, or optional parity, use `apb5_slave` from `rtl/amba/apb5/` – see the APB5 book.

6.1 Overview

The `apb_slave` module implements a full APB slave interface with integrated command and response buffering using GAXI skid buffers. It provides seamless protocol translation between the APB interface and a simple command/response interface, enabling easy integration of custom peripherals and register blocks into APB-based systems.

6.2 Module Declaration

```
module apb_slave #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    parameter int PROT_WIDTH      = 3,
    parameter int DEPTH           = 2,
    // Short Parameters
```

```
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Slave Interface
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

6.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width

Parameter	Type	Default	Description
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid buffer depth in **entries** (not a log2 exponent)

DEPTH is a literal entry count. `gaxi_skid_buffer` stores its payload in an unpacked array of `DEPTH` slots, so `DEPTH=2` gives two entries, not four. Supported values are `{2, 4, 6, 8}`; the shift-register storage is optimal at 2 and remains cheaper than a packed-vector implementation through 8. Odd values and values above 8 are not supported.

6.4 Ports

6.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

6.4.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	APB ready signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	APB write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	APB write data
s_apb_PSTRB	STRB_WIDTH	Input	APB write

Port	Width	Direction	Description
			strobes
s_apb_PPROT	PROT_WIDTH	Input	APB protection attributes
s_apb_PRDATA	DATA_WIDTH	Output	APB read data
s_apb_PSLVERR	1	Output	APB slave error

6.4.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid
cmd_ready	1	Input	Command ready (from backend)
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command write data
cmd_pstrb	STRB_WIDTH	Output	Command write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection attributes

6.4.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid
rsp_ready	1	Output	Response ready
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error

Port	Width	Direction	Description
			status

6.5 Functionality

6.5.1 APB Protocol Implementation

The module implements the complete APB slave protocol with a three-state finite state machine.

These are internal slave states, not APB bus phases. The APB SETUP and ACCESS phases are driven by the *master* (`PSEL` / `PENABLE`); a slave only observes them. `IDLE`, `BUSY`, and `WAIT` describe where this slave is in servicing the observed transfer:

1. **IDLE**: No transfer in flight. Waits for the SETUP-to-ACCESS edge (the cycle in which `PENABLE` first goes high while `PSEL` is high). Also drops any orphan response – see below.
2. **BUSY**: Command has been pushed to the backend; the bus is in its ACCESS phase with `PREADY` low (wait states). Waits for the backend response.
3. **WAIT**: `PREADY`, `PRDATA` and `PSLVERR` are driven for exactly one cycle, completing the ACCESS phase. Returns to IDLE unconditionally.

Mapping to the bus phases:

APB bus phase	Slave state	PREADY
IDLE (`PSEL=0`)	IDLE	0
SETUP (`PSEL=1`, `PENABLE=0`)	IDLE	0
ACCESS (`PSEL=1`, `PENABLE=1`), wait states	BUSY	0
ACCESS, final cycle	WAIT	1

6.5.2 Command/Response Flow

```
APB Master → APB Slave Interface → Command Buffer → Command Interface
→ Backend
                    ↓                          ↓                ↓
APB Master ← APB Slave Interface ← Response Buffer ← Response
Interface ← Backend
```

6.5.3 Buffering Architecture

The module uses two GAXI skid buffers for decoupling:

- **Command Buffer**: Stores incoming APB transactions
- **Response Buffer**: Stores backend responses for APB return

6.5.4 Key Features

- **APB4 Compliance**: Full protocol support including PSTRB and PPROT
- **Buffered Operation**: Command and response skid buffers prevent blocking
- **Flow Control**: Proper ready/valid handshaking on all interfaces
- **Error Handling**: PSLVERR propagation from backend to APB
- **Orphan-Response Guard**: Discards responses that arrive with no command outstanding

6.6 State Machine Operation

6.6.1 State Transitions

```
IDLE → BUSY: s_apb_PSEL && s_apb_PENABLE && !r_penable_prev &&
r_cmd_ready
BUSY → WAIT: r_rsp_valid (response available)
WAIT → IDLE: Automatic (1 cycle completion)
```

The `!r_penable_prev` term is a **rising-edge detect on `PENABLE`**. The command is captured on the SETUP-to-ACCESS transition only, so a multi-cycle ACCESS phase (the normal case, since this slave always inserts wait states) cannot re-issue the same command.

6.6.2 Orphan-Response Guard

APB is strictly one-outstanding: a command is issued on the IDLE-to-BUSY edge and its response is consumed in BUSY. A response sitting in the response skid buffer while the FSM is in IDLE therefore cannot belong to any command. In IDLE the FSM pops and discards it (asserting `r_rsp_ready` without accepting the data) and emits a simulation-only `$display` warning.

This matters because BUSY pairs commands with responses **by position, not by tag**. Without the guard, one stale entry permanently offsets the response stream: every subsequent read returns the previous read's data. Two ways this can happen in practice:

- A backend that emits a duplicate response for a single command.
- An `apb_slave_cdc` whose two clock domains were reset independently. The current CDC uses gray-pointer FIFOs specifically so it cannot fabricate a transfer this way – see apb_slave_cdc.md.

6.6.3 APB Transaction Timing

State	APB Signals	Internal Operation
IDLE	PREADY=0	Wait for rising edge of PENABLE; drop orphan responses
BUSY	PREADY=0	Command issued, wait for backend response
WAIT	PREADY=1, PRDATA/PSLVERR valid	Complete transaction

6.7 Timing Characteristics

6.7.1 Transaction Latency

This slave is **not** a zero-wait-state slave. `PREADY` is registered and is asserted no earlier than two `pclk` cycles after the edge on which `PENABLE` is first sampled high (one cycle IDLE-to-BUSY, one cycle BUSY-to-WAIT). The ACCESS phase therefore always contains at least two wait states, and a complete APB transfer takes at least four `pclk` cycles end to end. Backend response latency adds to this directly.

Characteristic	Value	Description
PENABLE rise to PREADY	≥2 clock cycles	Registered FSM path, best case
Minimum ACCESS wait states	2	Inherent to the three-state FSM
Minimum full transfer	≥4 clock cycles	SETUP + ACCESS with wait states
Response Processing	1+ clock cycles	Backend processing time, additive

6.7.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent, not characterized in this repository
Buffer Depth	2 entries	With default DEPTH=2
Outstanding Transactions	1	APB is non-pipelined; buffers absorb backend latency, they do not add concurrency

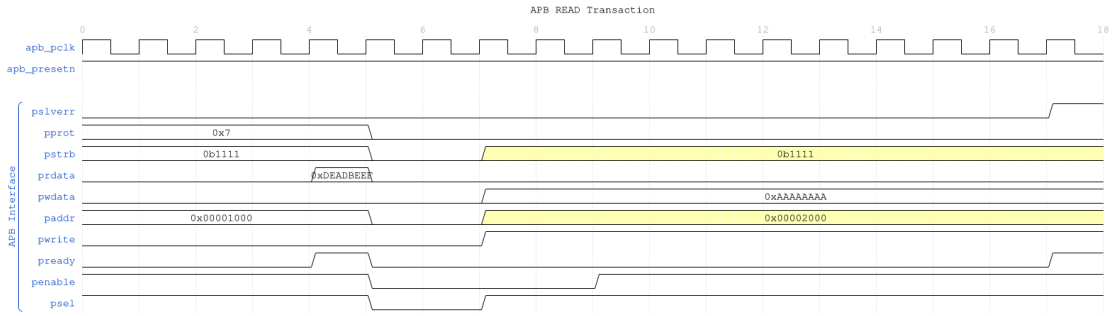
On throughput: APB is a non-pipelined protocol. The skid buffers decouple the APB FSM from backend latency; they do not allow a second APB transfer to start before the first completes. Peak throughput is therefore `DATA_WIDTH / (minimum transfer cycles × pclk period)`, not one transfer per cycle.

6.8 Waveforms

The following timing diagrams show the comprehensive APB slave behavior across 7 scenarios.

Known diagram defect: the WaveDrom generator stamps a fixed header string, `APB READ Transaction`, onto every scenario image regardless of direction. The signal traces themselves are correct – read `PWRITE` in the trace, not the header, to determine transaction direction. This affects the images under `docs/markdown/assets/WAVES/apb_slave/` and `.../apb_master/`.

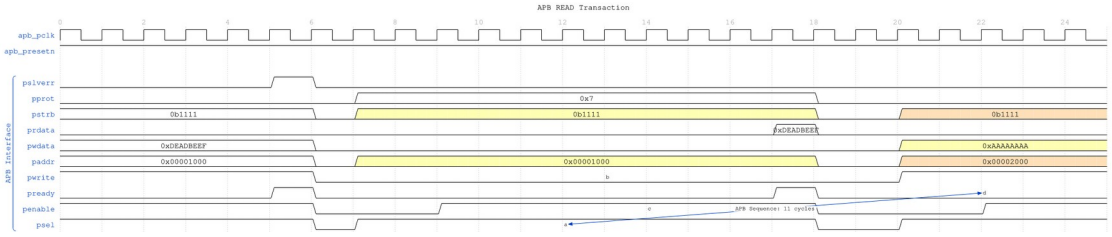
6.8.1 Scenario 1: Basic Write Transaction



APB Write

WaveJSON: apb_write_sequence_001.json

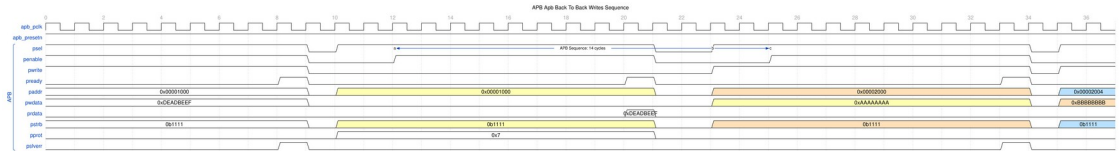
6.8.2 Scenario 2: Basic Read Transaction



APB Read

WaveJSON: apb_read_sequence_001.json

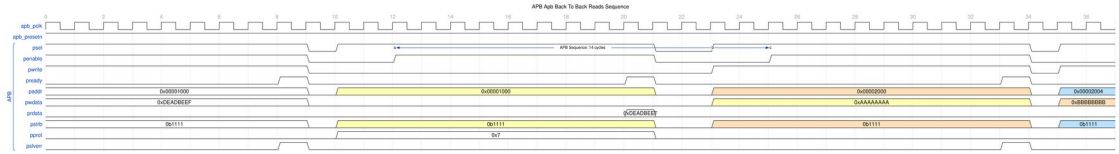
6.8.3 Scenario 3: Back-to-Back Writes



B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

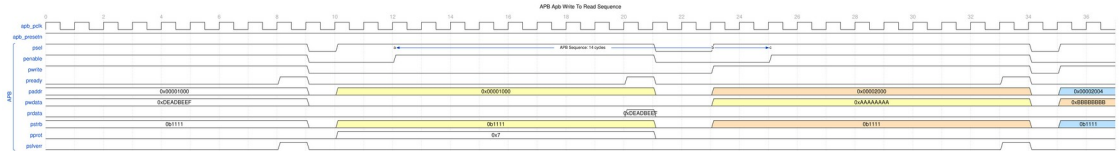
6.8.4 Scenario 4: Back-to-Back Reads



B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

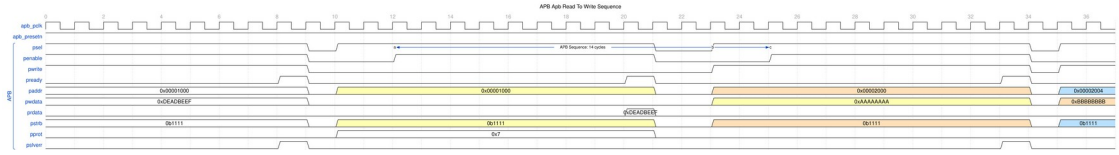
6.8.5 Scenario 5: Write-to-Read Transition



Write-to-Read

WaveJSON: apb_write_to_read_001.json

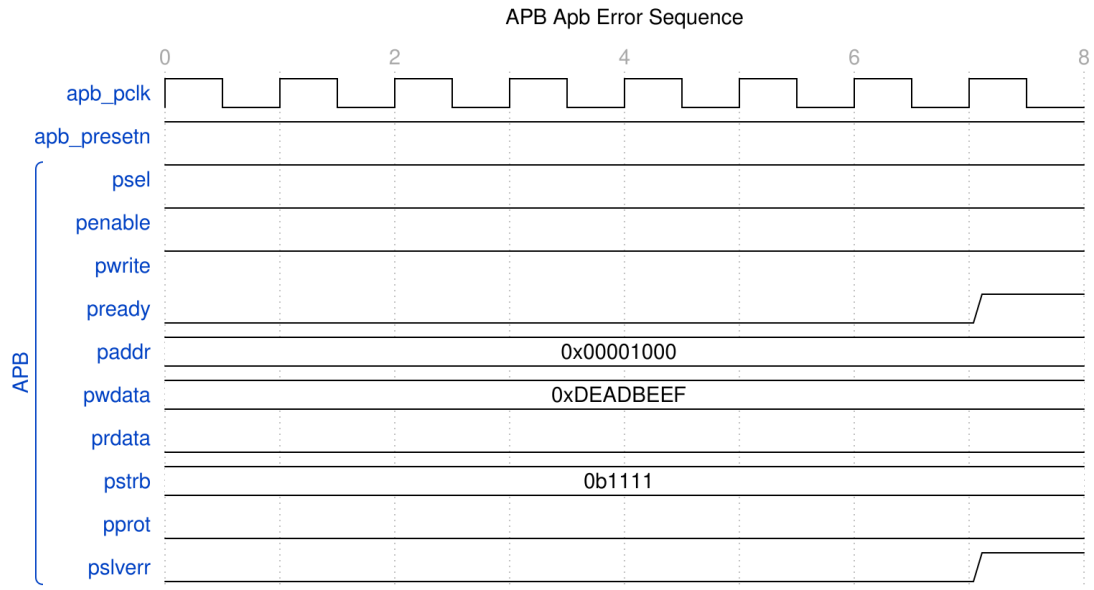
6.8.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

6.8.7 Scenario 7: Error Response



Error

WaveJSON: apb_error_001.json

6.9 Usage Examples

6.9.1 Basic Register Block Interface

```
apb_slave #(
    .ADDR_WIDTH(16),     // 64KB address space
    .DATA_WIDTH(32),
    .DEPTH(2)            // 4-entry buffers
) u_reg_slave (
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // APB interface from master
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
```

```
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface to register block
    .cmd_valid      (reg_cmd_valid),
    .cmd_ready      (reg_cmd_ready),
    .cmd_pwrite     (reg_write),
    .cmd_paddr      (reg_addr),
    .cmd_pwdata     (reg_wdata),
    .cmd_pstrb      (reg_strb),
    .cmd_pprot      (reg_prot),

    // Response interface from register block
    .rsp_valid      (reg_rsp_valid),
    .rsp_ready      (reg_rsp_ready),
    .rsp_prdata     (reg_rdata),
    .rsp_pslverr    (reg_error)
);
```

6.9.2 Register Block Backend Implementation

PADDR is a **byte** address. The example below holds 32-bit registers, so it drops the two low byte-lane bits and indexes with `cmd_addr[5:2]` to address 16 words. Adjust the slice if `DATA_WIDTH` is not 32.

```
// Simple register block backend
module register_block (
    input  logic        clk,
    input  logic        resetn,

    // Command interface
    input  logic        cmd_valid,
    output logic        cmd_ready,
    input  logic        cmd_write,
    input  logic [15:0] cmd_addr,
    input  logic [31:0] cmd_wdata,
    input  logic [3:0]  cmd_strb,

    // Response interface
    output logic        rsp_valid,
    input  logic        rsp_ready,
    output logic [31:0] rsp_rdata,
    output logic        rsp_error
);

    // Register array
    logic [31:0] registers [16];
```

```
    // Command processing
    always_ff @(posedge clk) begin
        if (cmd_valid && cmd_ready) begin
            if (cmd_write) begin
                // Write operation with byte strobes
                for (int i = 0; i < 4; i++) begin
                    if (cmd_strb[i]) begin
                        registers[cmd_addr[5:2]][i*8+:8] <=
cmd_wdata[i*8+:8];
                    end
                end
            end
        end
    end

    // Response generation
    always_ff @(posedge clk) begin
        if (!resetn) begin
            rsp_valid <= 1'b0;
            rsp_rdata <= 32'h0;
            rsp_error <= 1'b0;
        end else if (cmd_valid && cmd_ready) begin
            rsp_valid <= 1'b1;
            rsp_rdata <= cmd_write ? 32'h0 : registers[cmd_addr[5:2]];
            rsp_error <= (cmd_addr >= 16*4); // Address range check
        end else if (rsp_ready) begin
            rsp_valid <= 1'b0;
        end
    end

    assign cmd_ready = !rsp_valid || rsp_ready;

endmodule
```

6.9.3 Memory Interface Example

```
// APB slave for memory interface
apb_slave #(
    .ADDR_WIDTH(20),     // 1MB address space
    .DATA_WIDTH(32),
    .DEPTH(3)            // 8-entry buffers for memory latency
) u_mem_slave (
    .pclk         (mem_clk),
    .presetn      (mem_resetn),

    // APB interface
    .s_apb_*(apb_*),
```

```
    // Memory command interface
    .cmd_valid    (mem_cmd_valid),
    .cmd_ready    (mem_cmd_ready),
    .cmd_pwrite   (mem_write),
    .cmd_paddr    (mem_addr),
    .cmd_pwdata   (mem_wdata),
    .cmd_pstrb    (mem_strb),
    .cmd_pprot    (mem_prot),

    // Memory response interface
    .rsp_valid    (mem_rsp_valid),
    .rsp_ready    (mem_rsp_ready),
    .rsp_prdata   (mem_rdata),
    .rsp_pslverr  (mem_error)
);

// Memory controller interface
memory_controller u_mem_ctrl (
    .clk          (mem_clk),
    .resetn       (mem_resetn),

    // APB slave interface
    .cmd_*(mem_cmd_*),
    .rsp_*(mem_rsp_*),

    // Physical memory interface
    .mem_*(ddr_*)
);
```

6.10 Advanced Integration Patterns

6.10.1 Multi-Register Bank System

```
module multi_register_system (
    input logic clk, resetn,

    // APB interface
    input  logic        apb_psel,
    input  logic        apb_penable,
    output logic        apb_pready,
    input  logic [31:0] apb_paddr,
    input  logic        apb_pwrite,
    input  logic [31:0] apb_pwdata,
    input  logic [3:0]  apb_pstrb,
    input  logic [2:0]  apb_pprot,
    output logic [31:0] apb_prdata,
    output logic        apb_pslverr
```

```
);

    // APB slave with address-based routing
    apb_slave u_apb_slave (
        .pclk(clk),
        .presetn(resetn),
        .s_apb_*(apb_*),
        .cmd_*(cmd_*),
        .rsp_*(rsp_*)
    );

    // Address decoder for multiple register banks
    logic [2:0] bank_select;
    logic [2:0] bank_cmd_valid;
    logic [2:0] bank_rsp_valid;

    assign bank_select = cmd_paddr[31:29]; // Top 3 bits for bank
selection

    // Command distribution
    for (genvar i = 0; i < 3; i++) begin : gen_banks
        assign bank_cmd_valid[i] = cmd_valid && (bank_select == i);

        register_bank #(.BANK_ID(i)) u_bank (
            .clk(clk),
            .resetn(resetn),
            .cmd_valid(bank_cmd_valid[i]),
            .cmd_ready(bank_cmd_ready[i]),
            .cmd_*(cmd_*),
            .rsp_valid(bank_rsp_valid[i]),
            .rsp_*(rsp_*)
        );
    end

    // Response arbitration
    assign cmd_ready = |bank_cmd_ready;
    assign rsp_valid = |bank_rsp_valid;

endmodule
```

6.10.2 Error Handling and Status Reporting

```
// Enhanced backend with error handling
module enhanced_register_block (
    input  logic        clk,
    input  logic        resetn,
```

```
    // APB slave interface
    input  logic        cmd_valid,
    output logic        cmd_ready,
    input  logic        cmd_write,
    input  logic [15:0] cmd_addr,
    input  logic [31:0] cmd_wdata,
    input  logic [3:0]  cmd_strb,
    input  logic [2:0]  cmd_prot,

    output logic        rsp_valid,
    input  logic        rsp_ready,
    output logic [31:0] rsp_rdata,
    output logic        rsp_error
);

    // Error conditions
    logic addr_error, prot_error, access_error;

    assign addr_error = (cmd_addr >= 16*4);                    // Out
of range
    assign prot_error = (cmd_prot[0] == 1'b1);                 //
Instruction access
    assign access_error = addr_error || prot_error;

    // Register access with error checking
    always_ff @(posedge clk) begin
        if (cmd_valid && cmd_ready) begin
            if (cmd_write && !access_error) begin
                // Safe register write
                for (int i = 0; i < 4; i++) begin
                    if (cmd_strb[i]) begin
                        registers[cmd_addr[5:2]][i*8+:8] <=
cmd_wdata[i*8+:8];
                    end
                end
            end
        end
    end

    // Response with comprehensive error reporting
    always_ff @(posedge clk) begin
        if (!resetn) begin
            rsp_valid <= 1'b0;
            rsp_rdata <= 32'h0;
            rsp_error <= 1'b0;
        end else if (cmd_valid && cmd_ready) begin
            rsp_valid <= 1'b1;
```

```
            rsp_error <= access_error;

            if (access_error) begin
                rsp_rdata <= 32'hDEADBEEF;  // Error pattern
            end else begin
                rsp_rdata <= cmd_write ? 32'h0 :
registers[cmd_addr[5:2]];
            end
        end else if (rsp_ready) begin
            rsp_valid <= 1'b0;
        end
    end

    assign cmd_ready = !rsp_valid || rsp_ready;

endmodule
```

6.11 Performance Optimization

6.11.1 Buffer Depth Selection

`DEPTH` is an entry count and must be one of `{2, 4, 6, 8}`. Because APB allows only one outstanding transfer, extra depth does **not** buy concurrency – it only absorbs jitter in a backend that returns responses unevenly. `DEPTH=2` is correct for almost every backend.

Backend Type	Recommended DEPTH	Rationale
Register Block	2	Single-cycle response; deeper buffers are pure area
SRAM Controller	2	Latency is absorbed by wait states, not by buffering
External Memory	4	Variable response timing; small margin against stalls
Shared/arbitrated backend	4	Response return can be bursty under contention

6.11.2 Clock Domain Optimization

For different clock domains, use `apb_slave_cdc`:

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32)
) u_cdc_slave (
    // APB clock domain
    .pclk(apb_clk),
    .presetn(apb_resetn),
    .s_apb_*(apb_*),

    // Backend clock domain
    .aclk(backend_clk),
    .aresetn(backend_resetn),
    .cmd_*(backend_cmd_*),
    .rsp_*(backend_rsp_*)
);
```

The CDC clock/reset ports are `pclk`/`presetn` (APB side) and `aclk`/`aresetn` (backend side) – there is no `s_`/`m_` prefix on them.

6.12 Synthesis Considerations

6.12.1 Area Optimization

- Reduce DEPTH for area-constrained designs
- Share skid buffers across multiple slaves when possible
- Use synchronous reset for smaller area

6.12.2 Timing Optimization

- Register all command/response outputs
- Use appropriate buffer depths to meet timing
- Consider pipeline stages for high-frequency operation

6.12.3 Power Optimization

- Use clock-gated variant (`apb_slave_cg`) when available
- Implement conditional clock enables for inactive slaves
- Size buffers appropriately to minimize switching activity

6.13 Verification Notes

6.13.1 Protocol Compliance

- Verify APB setup and access phase timing
- Check PREADY assertion with valid PRDATA/PSLVERR
- Validate proper state machine operation

6.13.2 Buffer Verification

- Test buffer overflow/underflow conditions
- Verify command/response packet integrity
- Check flow control under various load conditions

6.13.3 Backend Integration

- Test various backend response latencies
- Verify error propagation and handling
- Check concurrent transaction handling

6.14 Related Modules

- **apb_slave_cg**: Clock-gated version for power optimization
- **apb_slave_cdc**: Clock domain crossing variant
- **apb_master**: Complementary APB master implementation
- **apb5_slave**: APB5 equivalent with `PWAKEUP`, `P*USER` and optional parity
- **gaxi_skid_buffer**: Underlying buffering infrastructure
- **apb_xbar_1to4** / **apb_xbar_2to4**: Generated APB crossbars for multi-slave systems

The `apb_slave` module provides a complete, high-performance solution for APB slave functionality with advanced buffering, full protocol compliance, and flexible backend integration capabilities.

6.15 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

7 APB Slave CDC

Module: `apb_slave_cdc.sv` **Location:** `rtl/amba/apb/` **Status:** √ Production Ready

---

7.1 Overview

The APB Slave CDC (Clock Domain Crossing) module provides a complete APB slave interface with integrated clock domain crossing between the APB (pclk) domain and an AXI/GAXI (aclk) domain. This enables safe integration of APB peripherals running at different clock frequencies.

7.1.1 Key Features

- √ **Safe CDC:** Gray-pointer asynchronous FIFOs, one per direction
- √ **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- √ **Independent Resets:** Either domain may be reset alone without corrupting the link
- √ **Full APB4 Support:** Complete AMBA 4 APB protocol compliance
- √ **Command/Response Interface:** Clean GAXI-style backend interface
- √ **Buffered Operation:** Integrated skid buffers for elastic storage

Protocol scope: APB4 only. For APB5 signalling use `apb5_slave_cdc` from `rtl/amba/apb5/` – see the APB5 book.

7.2 Module Interface

```
module apb_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH  = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH       = 2,
    // DEPRECATED / NO EFFECT -- retained only so existing
instantiations
    // still elaborate. See "CDC Implementation" below.
    parameter bit USE_2_PHASE_CDC = 1'b1
) (
    // Clock and Reset
    input  logic              aclk,
    input  logic              aresetn,
    input  logic              pclk,
    input  logic              presetn,
```

```
    // APB Slave Interface (pclk domain)
    input  logic               s_apb_PSEL,
    input  logic               s_apb_PENABLE,
    output logic               s_apb_PREADY,
    input  logic [AW-1:0]      s_apb_PADDR,
    input  logic               s_apb_PWRITE,
    input  logic [DW-1:0]      s_apb_PWDATA,
    input  logic [SW-1:0]      s_apb_PSTRB,
    input  logic [PW-1:0]      s_apb_PPROT,
    output logic [DW-1:0]      s_apb_PRDATA,
    output logic               s_apb_PSLVERR,

    // Command Interface (aclk domain)
    output logic               cmd_valid,
    input  logic               cmd_ready,
    output logic               cmd_pwrite,
    output logic [AW-1:0]      cmd_paddr,
    output logic [DW-1:0]      cmd_pwdata,
    output logic [SW-1:0]      cmd_pstrb,
    output logic [PW-1:0]      cmd_pprot,

    // Response Interface (aclk domain)
    input  logic               rsp_valid,
    output logic               rsp_ready,
    input  logic [DW-1:0]      rsp_prdata,
    input  logic               rsp_pslverr
);
```

7.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid-buffer depth **in entries** inside the wrapped `apb_slave`; also

Parameter	Type	Default	Description
			the floor for the CDC FIFO depth
USE_2_PHASE_CDC	bit	1	**Deprecated and ignored.** Has no effect on the generated logic

7.4 Clock Domains

7.4.1 APB Domain (pclk)

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

7.4.2 AXI Domain (aclk)

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

7.5 CDC Implementation

7.5.1 Structure

The module is an `apb_slave` in the `pclk` domain plus two independent asynchronous FIFOs:

FIFO	Direction	Write domain	Read domain	Payload width
`u_cmd_cdc_fifo`	Command	`pclk` / `presetn`	`aclk` / `aresetn`	`CPW = AW + DW + SW + PW + 1`
`u_rsp_cdc_fifo`	Response	`aclk` / `aresetn`	`pclk` / `presetn`	`RPW = DW + 1`

Both are `gaxi_fifo_async` instances with:

- **Pointer encoding:** gray-coded absolute read/write pointers
- **Synchronizer depth:** `N_FLOP_CROSS = 2` (two-flop synchronizer per crossed pointer)

- **FIFO depth:** `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` – a floor of 4 entries regardless of the `DEPTH` used for the internal skid buffers. Powers of two are preferred for the gray-pointer encoding.

There is no separate metastability-hardening option; two-flop synchronization is fixed at instantiation.

7.5.2 Maximum Clock Ratio

There is no maximum ratio between `pclk` and `aclk`. Gray-pointer FIFOs impose no relationship between the two clocks – either may be arbitrarily faster, slower, or phase-unrelated, and either may be stopped indefinitely. Stopping `aclk` simply stalls the command FIFO's read side; the APB side backpressures via `PREADY` held low and no data is lost.

Throughput, not correctness, is what the ratio affects: each transfer costs the usual two-flop synchronizer latency in each direction (roughly 2-3 destination clock edges per crossing), so a very slow `aclk` directly lengthens APB wait states.

7.5.3 Reset Behavior

`gaxi_fifo_async` resets each domain's own pointer **and** that domain's crossed copy of the remote pointer from that domain's local reset. Resetting one side in isolation therefore leaves that side's view self-consistent (both pointers zero, i.e. empty), and because the pointers are absolute positions rather than parity state, an independent reset cannot fabricate or swallow a transfer.

This is load-bearing: `presetn` and `aresetn` are genuinely separate reset domains in real integrations. For example, the `ddr2-char` harness pulses only the core-side reset on `CTRL.soft_reset` while the APB side stays up.

7.5.4 Why Not the Previous 2-Phase Handshake

`USE_2_PHASE_CDC` selected a toggle-based handshake in an earlier revision. It is now deprecated and ignored, and the parameter is retained only for source-compatibility with existing instantiations.

A 2-phase handshake encodes each transfer as a **toggle**. If the two domains are reset independently, the toggle parity desynchronizes and the link fabricates or drops exactly one transfer – permanently, because nothing ever re-syncs it. Paired with the `apb_slave` FSM, which pairs commands and responses by position rather than by tag, a single phantom transfer offsets the response stream by one forever: every read returns the previous read's data.

This was observed on the Nexys A7 `ddr2-char` board on 2026-07-19. Reading a single CSR eight times returned the previous register's value about three times before settling, while the non-CDC harness window was stable. The two mitigations now in place are the gray-pointer FIFOs described above and the orphan-response guard in `apb_slave` – see apb_slave.md.

7.5.5 Timing Constraints

The crossed signals are the gray-coded pointers and the FIFO memory read path. Standard practice applies:

- Declare `pclk` and `aclk` asynchronous to each other (`set_clock_groups -asynchronous`).
- Do not over-constrain the pointer synchronizer paths; the gray encoding tolerates a one-bit-at-a-time skew by construction.

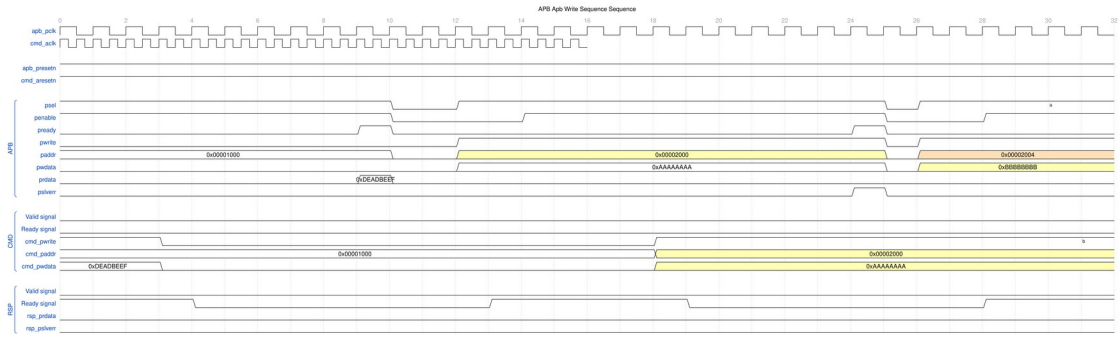
7.6 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

7.6.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from pclk domain to aclk domain:



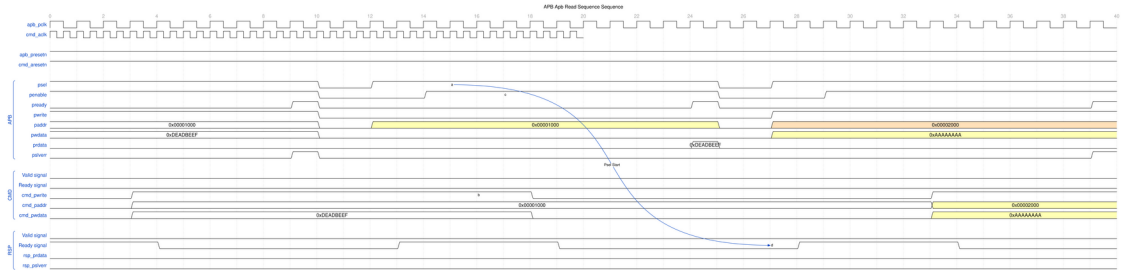
Write CDC

WaveJSON: apb_write_sequence_001.json

Key Observations: - APB transaction in pclk domain - CMD transaction crosses to aclk domain - Note the CDC latency between domains

7.6.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from aclk to pclk domain:

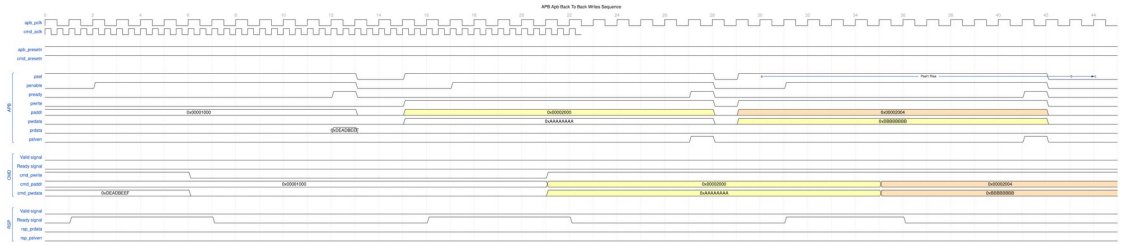


Read CDC

WaveJSON: apb_read_sequence_001.json

Key Observations: - APB read request in pclk domain - CMD crosses to aclk domain - RSP crosses back to pclk domain - Complete round-trip CDC visible

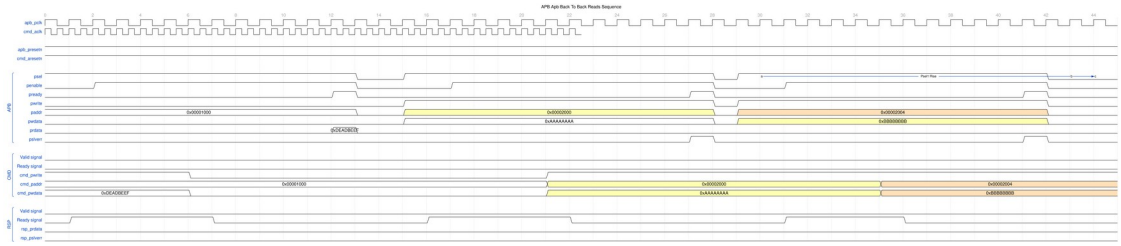
7.6.3 Scenario 3: Back-to-Back Writes with CDC



B2B Writes CDC

WaveJSON: apb_back_to_back_writes_001.json

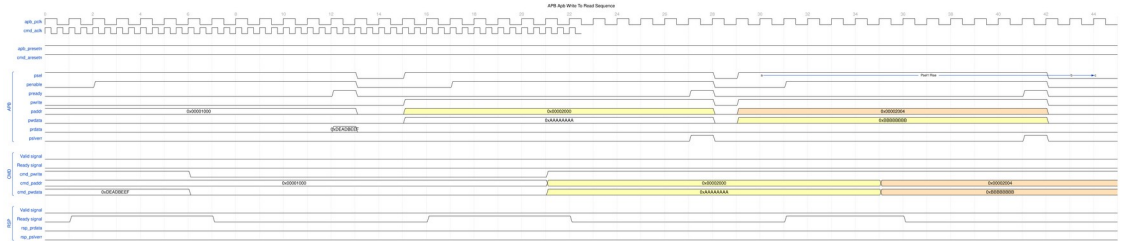
7.6.4 Scenario 4: Back-to-Back Reads with CDC



B2B Reads CDC

WaveJSON: apb_back_to_back_reads_001.json

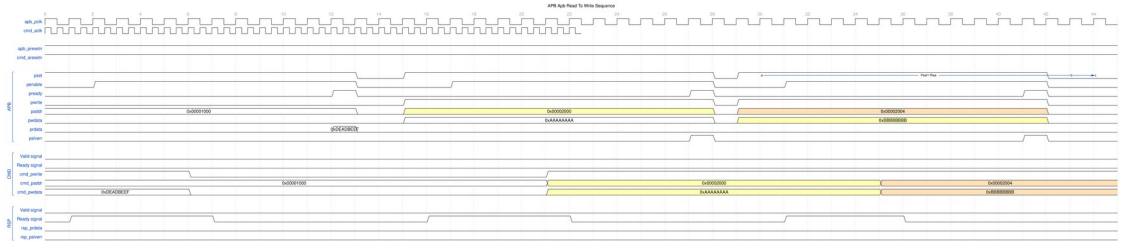
7.6.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: apb_write_to_read_001.json

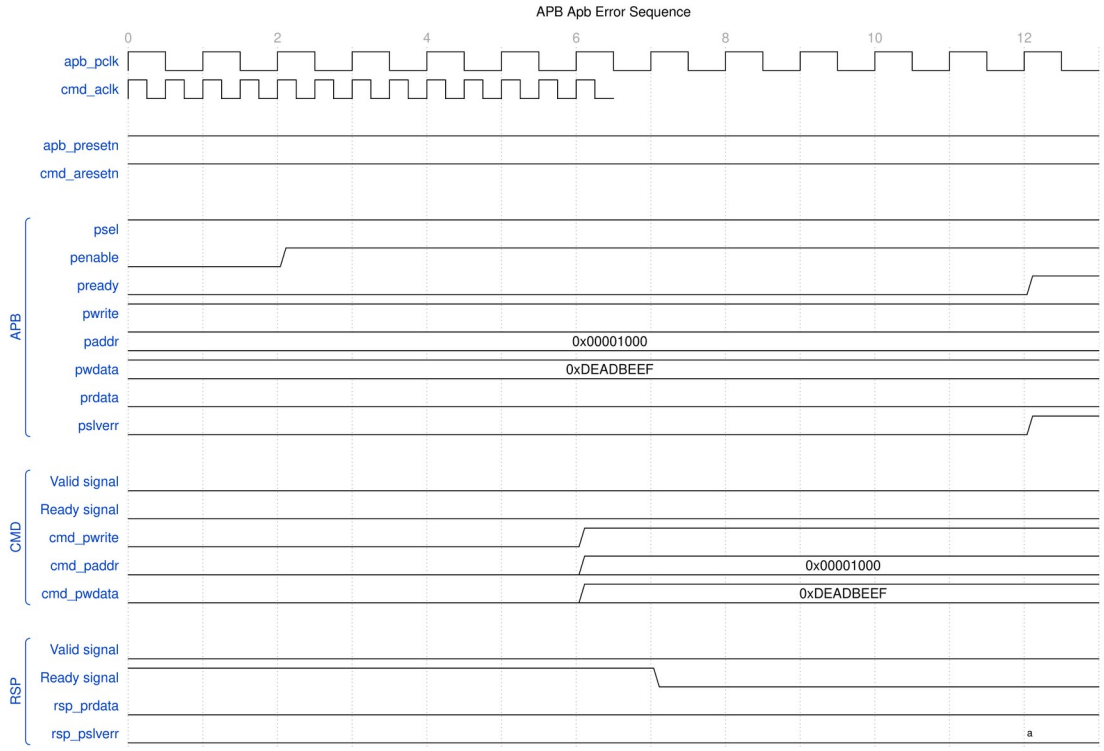
7.6.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: apb_read_to_write_001.json

7.6.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: apb_error_001.json

7.7 Usage Example

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // AXI clock domain
    .aclk         (axi_clk),
    .aresetn      (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL     (apb_psel),
```

```
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface (aclk domain)
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),

    // Response interface (aclk domain)
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

7.8 References

- **APB Slave:** apb_slave.md
- **Clock-Gated Variant:** apb_slave_cdc_cg.md
- **APB5 Equivalent:** apb5_slave_cdc.md
- **CDC FIFO:** `rtl/amba/gaxi/gaxi_fifo_async.sv`
- **Source:** `rtl/amba/apb/apb_slave_cdc.sv`
- **Tests:** `val/amba/test_apb_slave_cdc.py`
- **WaveDrom Test:** `val/amba/test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom`

Last Updated: 2026-07-19

7.9 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

8 APB Slave with CDC (Clock-Gated)

Module: `apb_slave_cdc_cg.sv` **Base Module:** apb_slave_cdc **Location:** `rtl/amba/apb/`
Status: √ Production Ready

8.1 Quick Reference

This is the **clock-gated variant** of apb_slave_cdc.

For the clock-gating architecture and the underlying gate cell, see:

→ Clock-Gated Variants Guide and **→ amba_clock_gate_ctrl**

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

8.2 Summary

The `apb_slave_cdc_cg` module adds power optimization to `apb_slave_cdc` through activity-based clock gating. **Both clock domains are gated independently** – the module instantiates two `amba_clock_gate_ctrl` blocks, one on `pclk` and one on `aclk`, sharing the same configuration inputs.

Structurally it is a sibling of `apb_slave_cdc` rather than a wrapper around it: it re-instantiates the same `apb_slave` plus the same pair of `gaxi_fifo_async` CDC FIFOs, but drives them from the gated clocks. The CDC behaviour described in apb_slave_cdc.md – gray pointers, `N_FLOP_CROSS=2`, independent-reset safety, no maximum clock ratio – applies unchanged.

While a domain is gated, that domain's `ready` outputs are forced low so no handshake can complete against a stopped clock.

- √ **Same Functionality:** 100% equivalent to base module
- √ **Dual-Domain Gating:** Separate gate cell per clock domain
- √ **Runtime Control:** Gating is enabled and tuned by input signals, not parameters

- ✓ **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

---

8.3 Additional Parameters

In addition to all apb_slave_cdc parameters:

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter, shared by both domains

`USE_2_PHASE_CDC` is inherited from the base module and is likewise **deprecated and ignored**. There is no `ENABLE_CLOCK_GATING` parameter and no per-domain `CG_GATE_*` parameters.

8.4 Additional Ports

In addition to all apb_slave_cdc ports:

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Global clock-gate enable for both domains. 0 = never gate
`cfg_cg_idle_count`	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating
`pclk_cg_gating`	1	Output	Asserted while the `pclk` domain is gated
`pclk_cg_idle`	1	Output	Asserted while the `pclk` domain buffers are empty
`aclk_cg_gating`	1	Output	Asserted while the `aclk` domain is gated
`aclk_cg_idle`	1	Output	Asserted while the `aclk` domain buffers are empty

8.5 Quick Usage

```
apb_slave_cdc_cg #(
    // Base module parameters (see apb_slave_cdc.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .DEPTH               (2),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),
    .aclk              (core_clk),
    .aresetn           (core_resetn),

    // Clock gating control (applies to both domains)
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),

    // Per-domain gating status
    .pclk_cg_gating    (pclk_gated),
    .pclk_cg_idle      (pclk_idle),
    .aclk_cg_gating    (aclk_gated),
    .aclk_cg_idle      (aclk_idle),

    // ... all other ports same as apb_slave_cdc
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging CDC waveforms. Two independently gated clocks make cross-domain timing very hard to read.

8.6 Documentation

- **Base Module Functionality:** apb_slave_cdc.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

8.7 Navigation

- **← Back to APB Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

9 APB Slave Interface (Clock-Gated)

Module: `apb_slave_cg.sv` **Base Module:** apb_slave **Location:** `rtl/amba/apb/` **Status:** √ Production Ready

---

9.1 Overview

The `apb_slave_cg` module is a clock-gated variant of apb_slave that adds comprehensive power optimization capabilities through activity-based clock gating.

9.1.1 Key Differences from Base Module

- √ **Activity-Based Clock Gating:** Gates `pclk` when the interface has been idle
- √ **Runtime Configuration:** Enable and idle threshold are input signals, not parameters
- √ **Gating Status Output:** `apb_clock_gating` reports when the clock is gated
- √ **Zero Functional Impact:** Maintains 100% functional equivalence with base module

The module is a thin wrapper: one `amba_clock_gate_ctrl` instance produces `gated_pclk`, which feeds an otherwise unmodified `apb_slave`. There is a **single** gate cell – there are no separate data-path and control-path gating domains.

All other functionality is identical to the base module. See apb_slave.md for complete functional specification.

9.2 When to Use Clock-Gated Variant

Use `apb_slave_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use base module (`apb_slave`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

---

9.3 Clock Gating Parameters

In addition to all parameters from apb_slave, this module adds:

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter; bounds the maximum programmable idle threshold

That is the only additional parameter. Gating is not parameterized on or off, and there are no per-domain gating parameters – both are controlled at runtime.

9.4 Clock Gating Ports

In addition to all ports from apb_slave:

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Global clock-gate enable. 0 = never gate (identical to base module)
`cfg_cg_idle_count`	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating the clock
`apb_clock_gating`	1	Output	Asserted while the internal clock is gated

9.5 Usage Examples

9.5.1 Example 1: Aggressive Gating (Bursty Traffic)

```
apb_slave_cg #(
    // Base parameters (see apb_slave.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
```

```
    .DEPTH               (2),
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd4),   // gate quickly after 4 idle cycles
    .apb_clock_gating  (cg_active),
    // ... connect signals same as base module
);
```

9.5.2 Example 2: Conservative Gating

```
apb_slave_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd15),  // wait longer; fewer gate/ungate
events
    .apb_clock_gating  (cg_active),
    // ... connect signals same as base module
);
```

Note that `cfg_cg_idle_count` is bounded by `CG_IDLE_COUNT_WIDTH`. With the default width of 4, the largest usable threshold is 15 cycles; widen the parameter if a longer idle window is required.

9.5.3 Example 3: Clock Gating Disabled (Functional Verification)

```
apb_slave_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b0),   // never gate
    .cfg_cg_idle_count (4'd0),
    // ... connect signals same as base module
);
```

Note: With `cfg_cg_enable = 0`, this module is functionally identical to the base module.

9.6 Clock Gating Architecture

9.6.1 Single Gating Domain

There is one `amba_clock_gate_ctrl` instance producing one gated clock, which feeds the entire wrapped `apb_slave`. There is no separate data-path/control-path split.

9.6.2 Wake-Up Condition

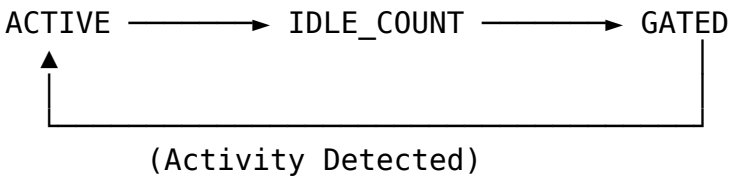
The wrapper holds the clock ungated while any of the following is true, sampled one cycle earlier into an internal `r_wakeup` register:

```
s_apb_PSEL || s_apb_PENABLE || cmd_valid || rsp_valid
```

That is: an APB transfer selected or in its ACCESS phase, a command still presented to the backend, or a response still pending. Gating engages `cfg_cg_idle_count + 1` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after all four terms go low, because APB adds two register stages ahead of the ICG enable.

9.6.3 Gating State Machine

The gate cell follows the standard three-state sequence:




```
States:
- ACTIVE:     Clock enabled, monitoring activity
- IDLE_COUNT: Counting cfg_cg_idle_count cycles before gating
- GATED:      Clock disabled, waiting for activity
```

9.6.4 Wake-Up Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This wrapper registers activity into its own `r_wakeup` flop and then hands it to `amba_clock_gate_ctrl`, which registers it again, so APB is a **two-stage** family. The first gated-clock rising edge available to the wrapped `apb_slave` therefore arrives **3 cycles** after activity asserts.

`clock_gate_ctrl` itself adds no flop: `w_gate_enable` is combinational from `wakeup` into the ICG enable, so the third cycle is the released clock edge rather than a further register stage.

`cfg_cg_idle_count` controls how long the clock stays running *after* traffic stops, not how long wake-up takes – a larger value means fewer gate/ungate transitions, not slower wake-up.

9.7 Power Savings

Clock gating removes the switching power of the wrapped `apb_slave` during the gated window. The achievable saving is therefore bounded by the fraction of time the interface is idle, and by whether the target technology maps the enable onto a real clock-gating cell (an ASIC ICG, a Xilinx `BUFGCE`, an Intel `ALTCLKCTRL`) rather than a data-path enable.

No power measurements have been taken for this module in this repository. Any specific percentage would be a guess. If a power number is needed, run vendor power analysis on the target device with a representative traffic switching activity file (SAIF/VCD) and compare against `apb_slave`.

9.7.1 Gating Status Signal

Signal	Width	Description
`apb_clock_gating`	1	Asserted while the internal clock is gated. Integrate over a window to measure the gated duty cycle

9.8 Verification Considerations

9.8.1 Clock Gating in Simulation

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
apb_slave_cg dut (
    .cfg_cg_enable (1'b0),   // Disable gating for functional debug
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Easier debug: a gated clock freezes internal state and reads like a hang

9.8.2 Power Analysis Verification

For power-specific verification:

1. **Enable gating** (`cfg_cg_enable = 1`) with a realistic `cfg_cg_idle_count`
2. **Monitor `apb_clock_gating`** to verify the expected gated duty cycle

3. **Vary traffic patterns** to test gating effectiveness
4. **Check wake-up timing** meets system requirements

9.9 Synthesis Considerations

9.9.1 FPGA Implementations

The gated clock is produced inside `amba_clock_gate_ctrl`. Whether it becomes a true gated clock or a fan-out of clock enables is a synthesis decision:

Xilinx: - Vivado typically converts the enable into `BUFGCE` or into per-flop clock enables - Confirm which happened in the post-synthesis netlist before claiming power savings - Verify with post-implementation power analysis

Intel (Altera): - Maps to `ALTCLKCTRL`; may need explicit vendor primitive instantiation - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

9.9.2 ASIC Implementations

- Work with foundry to select appropriate clock gating cells
- Integrated Clock Gating (ICG) cells provide best results
- Consider hold-time implications of clock gating
- Verify power intent with UPF (Unified Power Format)

9.10 Related Modules

- **apb_slave** - Base module (non-clock-gated)

9.11 See Also

- **Power Optimization Guide:** `docs/POWER_OPTIMIZATION_GUIDE.md`
- **Clock Gating Best Practices:** `docs/CLOCK_GATING_GUIDE.md`
- **AMBA Subsystem Overview:** `docs/markdown/RTLAmba/overview.md`

9.12 Navigation

- ← Back to Base Module
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

10 apb_slave_stub

A lightweight APB slave stub module that provides packed command/response interfaces for simplified testbench integration and system-level testing.

10.1 Overview

The `apb_slave_stub` module acts as a simple APB slave that converts APB protocol signals to packed command/response packets. It's designed for testbench use where a simple APB slave is needed without the full functionality of the standard `apb_slave` module. The packed interface simplifies integration with test responders and verification components.

10.2 Module Declaration

```
module apb_slave_stub #(
    parameter int DEPTH    = 4,
    parameter int DATA_WIDTH    = 32,
    parameter int ADDR_WIDTH    = 32,
    parameter int STRB_WIDTH    = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 4, // addr, data, strb, prot, pwrite
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1, // data, resp
    // Short Parameters
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
    // Clock and Reset
    input  logic                          pclk,
    input  logic                          presetn,

    // APB Slave Interface
    input  logic                          s_apb_PSEL,
    input  logic                          s_apb_PENABLE,
    input  logic [AW-1:0]                 s_apb_PADDR,
    input  logic                          s_apb_PWRITE,
    input  logic [DW-1:0]                 s_apb_PWDATA,
```

```
    input  logic [SW-1:0]               s_apb_PSTRB,
    input  logic [2:0]                  s_apb_PPROT,
    output logic [DW-1:0]               s_apb_PRDATA,
    output logic                        s_apb_PSLVERR,
    output logic                        s_apb_PREADY,

    // Command Packet Interface (packed)
    output logic                        cmd_valid,
    input  logic                        cmd_ready,
    output logic [CPW-1:0]              cmd_data,

    // Response Packet Interface (packed)
    input  logic                        rsp_valid,
    output logic                        rsp_ready,
    input  logic [RPW-1:0]              rsp_data
);
```

10.3 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Skid-buffer depth in **entries** (not a log2 exponent); must be one of `{2, 4, 6, 8}`
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

10.4 Ports

10.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

10.4.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	Peripheral select
s_apb_PENABLE	1	Input	Enable signal
s_apb_PADDR	AW	Input	Address bus
s_apb_PWRITE	1	Input	Write/read (1=write, 0=read)
s_apb_PWDATA	DW	Input	Write data
s_apb_PSTRB	SW	Input	Write strobe
s_apb_PPROT	3	Input	Protection attributes
s_apb_PRDATA	DW	Output	Read data
s_apb_PSLVERR	1	Output	Slave error
s_apb_PREADY	1	Output	Ready signal

10.4.3 Packed Command Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Ready for command packet
cmd_data	CPW	Output	Packed command (pwrite, pprot, pstrb, pwdata, paddr)

10.4.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for

Port	Width	Direction	Description
			response packet
rsp_data	RPW	Input	Packed response (resp, prdata)

10.5 Functional Description

10.5.1 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `pwdata` (DW bits): Write data - `paddr` (AW bits): Address

Response Packet Format (MSB to LSB): - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

10.5.2 Packet Format Is Not Symmetric With `apb_master_stub`

`apb_master_stub` carries two extra bits in each direction – `first` and `last`:

	`apb_slave_stub`	`apb_master_stub`
`CMD_PACKET_WIDTH`	`AW + DW + SW + 4`	`AW + DW + SW + 3 + 1 + 1 + 1`
Command fields	pwrite, pprot, pstrb, paddr, pwdata	last, first, pwrite, pprot, pstrb, paddr, pwdata
`RESP_PACKET_WIDTH`	`DW + 1`	`DW + 1 + 1 + 1`
Response fields	pslverr, prdata	last, first, pslverr, prdata

This is intentional, not a defect. The `first`/`last` bits exist so `apb_master_stub` can carry AXI4 burst framing through the AXI4-to-APB bridge; an APB slave has no burst to frame and so has no use for them.

The two stubs are **never** connected packed-side to packed-side. They face each other across the APB bus, and the APB bus itself has no `first`/`last` signals, so no incompatibility arises. Do not attempt to wire `apb_master_stub.cmd_data` directly into `apb_slave_stub` – the widths differ and the field alignment differs.

10.5.3 Difference From apb_slave

This stub is a simplified two-state FSM (IDLE, XFER_DATA) with combinational outputs. It does **not** implement the PENABLE rising-edge detect or the orphan-response guard present in apb_slave. Use apb_slave for anything synthesized into a real design, particularly behind a CDC.

10.5.4 Operation

1. APB master drives transaction on APB interface
2. Stub detects PSEL/PENABLE and packs command
3. Stub presents command packet on cmd_data with cmd_valid=1
4. Test responder reads command when cmd_ready=1
5. Test responder provides response on rsp_data with rsp_valid=1
6. Stub unpacks response and drives APB PRDATA/PSLVERR/PREADY

10.6 Usage Example

```
// Instantiate APB slave stub
apb_slave_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb_slave_stub (
    .pclk(clk),
    .presetn(rst_n),
    // APB interface from master/interconnect
    .s_apb_PSEL(apb_psel),
    .s_apb_PENABLE(apb_penable),
    .s_apb_PADDR(apb_paddr),
    .s_apb_PWRITE(apb_pwrite),
    .s_apb_PWDATA(apb_pwdata),
    .s_apb_PSTRB(apb_pstrb),
    .s_apb_PPROT(apb_pprot),
    .s_apb_PRDATA(apb_prdata),
    .s_apb_PSLVERR(apb_pslverr),
    .s_apb_PREADY(apb_pready),
    // Packed interface to test responder
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Respond to command (in testbench)
// When cmd_valid: unpack command, generate response
```

```
always_comb begin
    test_cmd_ready = 1'b1;  // Always ready
    if (test_cmd_valid) begin
        // Unpack command
        logic [AW-1:0] addr = test_cmd_data[AW-1:0];
        logic [DW-1:0] wdata = test_cmd_data[AW+DW-1:AW];
        logic pwrite = test_cmd_data[CPW-1];

        // Generate response
        test_rsp_data = {1'b0, read_memory(addr)};  // No error,
return data
        test_rsp_valid = 1'b1;
    end
end
```

10.7 Design Notes

10.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB slave emulation - Memory model integration - Verification component integration

For production use, consider the full-featured `apb_slave` module.

10.7.2 Response Generation

The stub expects immediate response from the test driver. For multi-cycle latency: - Test driver should buffer commands - Assert `rsp_valid` only when response is ready - Stub will hold PREADY low until response arrives

10.7.3 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for memory models and responders

10.8 Related Modules

- `apb_slave.sv` - Full-featured APB slave with independent interfaces
- `apb_master_stub.sv` - Companion APB master stub
- `apb_monitor.sv` - APB transaction monitoring

10.9 References

- **APB Protocol**: ARM IHI 0024C – AMBA APB Protocol Specification, Version 2.0 (APB4)
- **Full APB Slave**: apb_slave.md
- **APB Master Stub**: apb_master_stub.md
- **APB5 Equivalent**: apb5_slave_stub.md

10.10 Navigation

- **← Back to APB Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

11 apb_xbar_thin

A fully parameterized M×S APB crossbar switch providing full connectivity between multiple APB masters and slaves with weighted round-robin arbitration and runtime-programmable address decoding.

Module: `apb_xbar_thin.sv` **Location:** `projects/components/apb_xbar/rtl/` **Status:** √ Production Ready

11.1 Naming Note

Earlier revisions of this document described a module named `apb_xbar` with separate `MST_THRESHOLDS` and `SLV_THRESHOLDS` inputs, an internal `DEPTH` parameter, `apb_slave`/`apb_master` stub conversion, side queues, and a 3-cycle minimum latency.

No such module exists. The parameterized crossbar in this repository is `apb_xbar_thin`, and it has a materially different architecture – it is combinational and does no protocol conversion at all. This document has been rewritten against the actual RTL.

The other family, the fixed-configuration generated crossbars (`apb_xbar_1to1`, `apb_xbar_2to1`, `apb_xbar_1to4`, `apb_xbar_2to4`), *does* use `apb_slave` + `apb_master` conversion and is specified separately in apb_crossbar.md.

11.1.1 Choosing Between the Two Families

	`apb_xbar_thin`	Generated crossbars
Topology	Any M×S, set by parameter	Fixed per generated file
Address map	Runtime inputs, per-slave base/limit/enable	Compile-time, uniform 64 KB regions
Arbitration	Weighted round-robin	Plain round-robin
Protocol conversion	None – combinational passthrough	APB → cmd/rsp → APB
Buffering	None	Skid buffers on both sides
Added latency	Zero cycles	Multiple cycles (see apb_crossbar.md)
Combinational path	Master APB to slave APB, and `PREADY`/`PRDATA` back	Broken by registers
Best for	Sparse or reprogrammable maps, latency-critical paths	Timing closure at higher frequency

The word “thin” is the operative one: this module adds no pipeline stages, which makes it the lowest-latency choice and the hardest one to close timing on.

11.2 Module Declaration

```
module apb_xbar_thin #(
    // Number of APB masters (from the master)
    parameter int M = 2,
    // Number of APB slaves (to the dest)
    parameter int S = 4,
    // Address width
    parameter int ADDR_WIDTH = 32,
    // Data width
    parameter int DATA_WIDTH = 32,
    // Strobe width
    parameter int STRB_WIDTH = DATA_WIDTH/8,
    parameter int MAX_THRESH = 16,
    // local abbreviations
    parameter int DW    = DATA_WIDTH,
    parameter int AW    = ADDR_WIDTH,
```

```
    parameter int SW    = STRB_WIDTH,
    parameter int MTW   = $clog2(MAX_THRESH),
    parameter int MXMTW = M * MTW
) (
    input  logic                          pclk,
    input  logic                          presetn,

    // Slave enable for addr decoding
    input  logic [S-1:0]                  SLAVE_ENABLE,
    // Slave address base
    input  logic [S-1:0][ADDR_WIDTH-1:0] SLAVE_ADDR_BASE,
    // Slave address limit
    input  logic [S-1:0][ADDR_WIDTH-1:0] SLAVE_ADDR_LIMIT,
    // Thresholds for the Weighted Round Robin Arbiter
    input  logic [MXMTW-1:0]              THRESHOLDS,

    // Master interfaces - These are from the APB master
    input  logic [M-1:0]                  m_apb_psel,
    input  logic [M-1:0]                  m_apb_penable,
    input  logic [M-1:0]                  m_apb_pwrite,
    input  logic [M-1:0][2:0]             m_apb_pprot,
    input  logic [M-1:0][ADDR_WIDTH-1:0] m_apb_paddr,
    input  logic [M-1:0][DATA_WIDTH-1:0] m_apb_pwdata,
    input  logic [M-1:0][STRB_WIDTH-1:0] m_apb_pstrb,
    output logic [M-1:0]                  m_apb_pready,
    output logic [M-1:0][DATA_WIDTH-1:0] m_apb_prdata,
    output logic [M-1:0]                  m_apb_pslverr,

    // Slave interfaces - these are to the APB destinations
    output logic [S-1:0]                  s_apb_psel,
    output logic [S-1:0]                  s_apb_penable,
    output logic [S-1:0]                  s_apb_pwrite,
    output logic [S-1:0][2:0]             s_apb_pprot,
    output logic [S-1:0][ADDR_WIDTH-1:0] s_apb_paddr,
    output logic [S-1:0][DATA_WIDTH-1:0] s_apb_pwdata,
    output logic [S-1:0][STRB_WIDTH-1:0] s_apb_pstrb,
    input  logic [S-1:0]                  s_apb_pready,
    input  logic [S-1:0][DATA_WIDTH-1:0] s_apb_prdata,
    input  logic [S-1:0]                  s_apb_pslverr
);
```

Note the port naming convention: lowercase (`m_apb_psel`, not `m_apb_PSEL`), unlike `apb_slave` / `apb_master` and the generated crossbars.

11.3 Parameters

Parameter	Type	Default	Description
M	int	2	Number of APB masters (input side)
S	int	4	Number of APB slaves (output side)
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	APB write strobe width (derived)
MAX_THRESH	int	16	Sizes the per-master threshold field: `MTW = $clog2(MAX_THRESH)`

There is no `DEPTH` parameter. The module contains no buffers.

`MAX_THRESH` only sizes the field. The arbiter is instantiated with `MAX_LEVELS(16)` hardcoded, so changing `MAX_THRESH` changes the width of `THRESHOLDS` without changing the arbiter's internal credit range. Leave it at the default of 16 unless you have also reviewed `rtl/common/arbiter_round_robin_weighted.sv`.

11.4 Ports

11.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

11.4.2 Configuration Interface

All configuration is by **input signal**, not parameter, so the address map can be reprogrammed at runtime.

Port	Width	Direction	Description
SLAVE_ENABLE	S	Input	Per-slave enable bit; a disabled slave never matches

Port	Width	Direction	Description
SLAVE_ADDR_BASE	S × ADDR_WIDTH	Input	Inclusive base address for each slave
SLAVE_ADDR_LIMIT	S × ADDR_WIDTH	Input	Inclusive limit address for each slave
THRESHOLDS	M × MTW	Input	Per-**master** arbitration weights

`THRESHOLDS` is a single per-master vector shared by every slave arbiter. There is no separate `MST_THRESHOLDS`/`SLV_THRESHOLDS` pair and no way to weight masters differently on different slaves – the same vector is wired to all `S` arbiter instances.

11.4.3 Master Interfaces (Input Side)

These are the crossbar's upstream ports; the crossbar behaves as an APB *slave* on them. They are named `m_apb_*` because they connect to external APB masters.

Port	Width	Direction	Description
m_apb_psel	M	Input	Master select signals
m_apb_penable	M	Input	Master enable signals
m_apb_pwrite	M	Input	Master write/read indicators
m_apb_pprot	M × 3	Input	Master protection attributes
m_apb_paddr	M × ADDR_WIDTH	Input	Master addresses
m_apb_pwdata	M × DATA_WIDTH	Input	Master write data
m_apb_pstrb	M × STRB_WIDTH	Input	Master write strobes
m_apb_pready	M	Output	Ready returned to each master

Port	Width	Direction	Description
m_apb_prdata	M × DATA_WIDTH	Output	Read data returned to each master
m_apb_pslverr	M	Output	Error returned to each master

11.4.4 Slave Interfaces (Output Side)

These are the crossbar's downstream ports; the crossbar behaves as an APB *master* on them. They are named `s_apb_*` because they connect to external APB slaves.

Port	Width	Direction	Description
s_apb_psel	S	Output	Slave select signals
s_apb_penable	S	Output	Slave enable signals
s_apb_pwrite	S	Output	Slave write/read indicators
s_apb_pprot	S × 3	Output	Slave protection attributes
s_apb_paddr	S × ADDR_WIDTH	Output	Slave addresses
s_apb_pwdata	S × DATA_WIDTH	Output	Slave write data
s_apb_pstrb	S × STRB_WIDTH	Output	Slave write strobes
s_apb_pready	S	Input	Ready from each slave
s_apb_prdata	S × DATA_WIDTH	Input	Read data from each slave
s_apb_pslverr	S	Input	Error from each slave

11.4.5 A Note on the `m_/s_` Prefixes

This convention is the opposite of the AMBA interconnect convention, in which an interconnect's *slave* port faces an upstream master and its *master* port faces a downstream slave. Here the prefix names **what is attached**, not what the crossbar acts as:

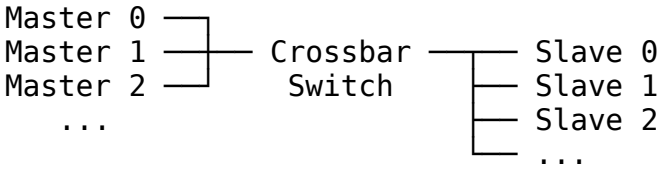
- `m_apb_*` = the ports that external **m**asters attach to (crossbar acts as slave)
- `s_apb_*` = the ports that external **s**laves attach to (crossbar acts as master)

The port directions in the tables above are unambiguous; use them, not the prefix, when wiring.

11.5 Architecture

11.5.1 Crossbar Topology

The crossbar provides full connectivity between all masters and slaves:



11.5.2 Key Components

1. **Address Decoders** (S instances): combinational per-slave, per-master range match
2. **Weighted Round-Robin Arbiters** (S instances): one `arbiter_round_robin_weighted` per slave
3. **Grant-ACK Registers** (S × M flops): the only sequential logic in the datapath
4. **Slave-side Multiplexers** (S instances): select the granted master's APB signals
5. **Master-side Demultiplexers** (M instances): return `PREADY`/`PRDATA`/`PSLVERR` from the granted slave

There are no stubs, no command queues, no side queues, and no response FIFOs.

11.5.3 Data Flow

```
APB Masters → Address Decode → Per-Slave Arbitration → Slave Mux → APB Slaves

APB Slaves → Master Demux (by grant) → APB Masters
```

Both directions are combinational. The only registers are the per-slave, per-master grant-ACK flops.

11.6 Functionality

11.6.1 Address Decoding

Each slave is assigned an address range defined by:

- **SLAVE_ADDR_BASE[s]**: inclusive starting address for slave s
- **SLAVE_ADDR_LIMIT[s]**: inclusive ending address for slave s
- **SLAVE_ENABLE[s]**: enable bit for slave s

Address matching is fully combinational and evaluated for every (slave, master) pair:

```
master_sel[s][m] = m_apb_psel[m] && SLAVE_ENABLE[s] &&
                   (m_apb_paddr[m] >= SLAVE_ADDR_BASE[s]) &&
                   (m_apb_paddr[m] <= SLAVE_ADDR_LIMIT[s]);
```

Because base and limit are independent runtime inputs, sparse and non-uniform maps are supported – unlike the generated crossbars, which use fixed uniform 64 KB regions.

Overlapping ranges are not detected. If two enabled slaves both match an address, both arbiters will see a request and both slaves will be selected. The master-side demultiplexer then returns whichever match it encounters last in its loop. Ensure the programmed ranges are disjoint.

11.6.2 Weighted Round-Robin Arbitration with ACK Mode

Each slave has an independent arbiter:

```
arbiter_round_robin_weighted #(
    .MAX_LEVELS  (16),
    .CLIENTS     (M),
    .WAIT_GNT_ACK(1)
) arbiter_inst (
    .max_thresh  (THRESHOLDS),
    .request     (master_sel[s]),
    .grant_valid (arb_gnt_valid[s]),
    .grant       (arb_gnt[s]),
    .grant_id    (arb_gnt_id[s]),
    .grant_ack   (arb_gnt_ack[s])
);
```

- **WAIT_GNT_ACK=1**: the grant is held until acknowledged, so it persists for the whole APB transfer and the transaction is atomic.

- **Grant ACK** is registered and asserted the cycle after the transfer completes on that slave:

```
arb_gnt_ack[s][m] <= arb_gnt[s][m] && s_apb_pready[s] &&
                     s_apb_psel[s]  && s_apb_penable[s];
```

- **Weighting**: `THRESHOLDS` gives each master a credit allowance, so a higher-weighted master wins more often across a rotation while still being starvation-free.

11.7 Timing Characteristics

11.7.1 Latency

`apb_xbar_thin` adds **zero cycles** of latency. The master's `PSEL`, `PENABLE`, `PADDR`, `PWDATA`, `PSTRB` and `PPROT` reach the selected slave combinationally, and `PREADY`, `PRDATA` and `PSLVERR` return combinationally. An uncontended transfer completes in exactly the downstream slave's own transfer time.

The cost is timing, not cycles:

Path	Description
Master `PADDR` → decode → arbiter request → grant → slave mux → `s_apb_paddr`	Forward combinational path
`s_apb_pready` → master demux (indexed by grant) → `m_apb_pready`	Return combinational path

Both paths scale with M and S. On FPGA targets this is normally the critical path of any design instantiating a large `apb_xbar_thin`. If timing does not close, use a generated crossbar from apb_crossbar.md, which breaks these paths with registers, or register the crossbar's boundaries externally.

11.7.2 Arbitration Turnaround

Because `grant_ack` is registered, the arbiter releases a grant one cycle after the transfer completes. Back-to-back transfers from *different* masters to the same slave therefore see one dead cycle between them. Transfers to *different* slaves proceed fully in parallel.

11.7.3 Throughput

Metric	Value	Conditions
Added latency	0 cycles	Any configuration

Metric	Value	Conditions
Concurrent transfers	Up to min(M, S)	Distinct slaves, no address conflict
Same-slave turnaround	1 dead cycle	Master change; registered grant ACK

No synthesis frequency or area figures are published for this module – none have been measured in this repository.

11.8 Usage Example

11.8.1 3 Masters, 4 Slaves with a Sparse Address Map

```
localparam int M = 3;
localparam int S = 4;

logic [S-1:0]             slave_enable;
logic [S-1:0][31:0]       slave_base;
logic [S-1:0][31:0]       slave_limit;
logic [M-1:0][3:0]        thresholds;

always_comb begin
    // Sparse, non-uniform map -- ranges need not be contiguous or
equal-sized
    slave_enable = 4'b1111;

    slave_base[0]  = 32'h4000_0000;  slave_limit[0] = 32'h4000_0FFF;
//  4 KB
    slave_base[1]  = 32'h4001_0000;  slave_limit[1] = 32'h4001_FFFF;
// 64 KB
    slave_base[2]  = 32'h5000_0000;  slave_limit[2] = 32'h500F_FFFF;
//  1 MB
    slave_base[3]  = 32'h6000_0000;  slave_limit[3] = 32'h6000_00FF;
// 256 B

    // Per-master weights: master 0 gets the largest share
    thresholds[0] = 4'hC;
    thresholds[1] = 4'h4;
    thresholds[2] = 4'h4;
end

apb_xbar_thin #(
    .M          (M),
    .S          (S),
```

```
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .MAX_THRESH (16)
) u_xbar (
    .pclk             (apb_clk),
    .presetn          (apb_resetn),

    .SLAVE_ENABLE     (slave_enable),
    .SLAVE_ADDR_BASE  (slave_base),
    .SLAVE_ADDR_LIMIT (slave_limit),
    .THRESHOLDS       (thresholds),

    // Upstream: external APB masters
    .m_apb_psel       (mst_psel),
    .m_apb_penable    (mst_penable),
    .m_apb_pwrite     (mst_pwrite),
    .m_apb_pprot      (mst_pprot),
    .m_apb_paddr      (mst_paddr),
    .m_apb_pwdata     (mst_pwdata),
    .m_apb_pstrb      (mst_pstrb),
    .m_apb_pready     (mst_pready),
    .m_apb_prdata     (mst_prdata),
    .m_apb_pslverr    (mst_pslverr),

    // Downstream: external APB slaves
    .s_apb_psel       (slv_psel),
    .s_apb_penable    (slv_penable),
    .s_apb_pwrite     (slv_pwrite),
    .s_apb_pprot      (slv_pprot),
    .s_apb_paddr      (slv_paddr),
    .s_apb_pwdata     (slv_pwdata),
    .s_apb_pstrb      (slv_pstrb),
    .s_apb_pready     (slv_pready),
    .s_apb_prdata     (slv_prdata),
    .s_apb_pslverr    (slv_pslverr)
);
```

11.8.2 Runtime Address Map Reprogramming

Because SLAVE_ADDR_BASE, SLAVE_ADDR_LIMIT and SLAVE_ENABLE are inputs, a control register block can rewrite the map at runtime:

```
always_ff @(posedge apb_clk or negedge apb_resetn) begin
    if (!apb_resetn) begin
        slave_enable <= '0;                 // decode nothing until
programmed
    end else if (cfg_write) begin
        case (cfg_addr)
```

```
                12'h000: slave_base[0]  <= cfg_wdata;
                12'h004: slave_limit[0] <= cfg_wdata;
                12'h008: slave_enable   <= cfg_wdata[S-1:0];
                // ... one base/limit pair per slave
                default: ;                       // no change
            endcase
        end
    end
```

Reprogram only while idle. The decode is combinational with no shadowing, so changing a base or limit mid-transfer can re-decode a transaction that has already been granted, moving `PSEL` to a different slave partway through and breaking the transfer. Quiesce all masters, or hold `SLAVE_ENABLE` low, before rewriting the map.

11.9 Known Limitations

11.9.1 Unmapped Addresses Stall the Bus

There is no default slave and no decode-error response. If no enabled slave matches, `master_sel[s][m]` is low for every `s`, no arbiter raises a grant, and the master-side demultiplexer leaves `m_apb_pready[m]` at its default of `1'b0`. `PREADY` is never asserted and **the transfer hangs indefinitely.** There is no timeout.

This is the same limitation as the generated crossbars – see apb_crossbar.md.

Mitigation: hold `SLAVE_ENABLE` such that the programmed ranges cover every address a master can emit, or place an address filter with an error slave upstream of the crossbar.

11.9.2 Other Limitations

Limitation	Detail
Overlapping ranges	Not detected; multiple slaves may be selected simultaneously
`MAX_THRESH`	Sizes `THRESHOLDS` only; the arbiter's `MAX_LEVELS` is hardcoded to 16
Shared weights	One `THRESHOLDS` vector drives all `S` arbiters; per-slave weighting is not possible
Combinational paths	Forward and return paths both scale with M and S;

Limitation	Detail
	usually the design's critical path
No monitoring	No `monbus` instrumentation, transaction counters, or timeout detection

11.10 Synthesis Considerations

11.10.1 Area

Area is dominated by the address comparators and the multiplexers:

Structure	Scaling
Address comparators	O(M × S × ADDR_WIDTH) – two comparators per pair
Arbiters	O(S) instances, each O(M)
Grant-ACK registers	O(M × S) flops
Slave-side muxes	O(S × M × (ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + 6))
Master-side demuxes	O(M × S × (DATA_WIDTH + 2))

The `M × S` terms dominate quickly. A large configuration (say M=6, S=12 at DATA_WIDTH=64) is substantially more logic than the numbers of masters and slaves suggest; budget for it before committing to a topology.

11.10.2 Timing

- Register the crossbar's boundaries externally if the combinational paths do not close
- Reduce M and S to the minimum required – both forward and return paths scale with them
- Consider a hierarchy of small crossbars instead of one large one
- If timing still fails, switch to a generated crossbar, which is registered by construction

11.11 Verification

11.11.1 Formal

SymbiYosys proofs are checked in at `formal/apb_xbar/apb_xbar_thin/`, with `prove` and `cover` tasks.

11.11.2 Simulation

`apb_xbar_thin` has no dedicated CocoTB testbench. The four generated variants in `projects/components/apb_xbar/dv/tests/` do, but they exercise the *other* architecture and provide no coverage of this module. Weigh that when selecting between the two families.

11.11.3 Suggested Coverage

- All M×S path combinations
- Address decoding at every range boundary (base, base-1, limit, limit+1)
- `SLAVE_ENABLE` masking
- Arbitration fairness and weighting under sustained contention
- Grant persistence across a slave's wait states
- Reset behaviour with a transfer in flight

11.12 Related Modules

- **apb_xbar_1to1** / **2to1** / **1to4** / **2to4**: fixed-configuration generated crossbars (apb_crossbar.md)
- **apb_master**: APB master, used by the generated crossbars (apb_master.md)
- **apb_slave**: APB slave, used by the generated crossbars (apb_slave.md)
- **arbiter_round_robin_weighted**: the per-slave arbiter instantiated here
- **arbiter_round_robin**: plain round-robin used by the generated crossbars
- **apb_monitor**: APB protocol monitoring (apb_monitor.md)

Dependencies:

- `rtl/common/arbiter_round_robin_weighted.sv`
- `rtl/common/arbiter_priority_encoder.sv` (arbiter dependency)

Last Updated: 2026-07-19

11.13 Navigation

- ← Back to APB Index
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index